Diverse Initializations for Vector Ensembles in Representation-aware Text-Attributed Graph Encoding with Large Language Models

ARTICLE INFO

Keywords:
Text-Attributed Graphs
Large Language Models
Graph Neural Networks
Ensemble Learning
LLM on Graph
Parameter-efficient fine-tuning
Data privacy

ABSTRACT

Learning on text-attributed graphs requires combining rich node text with graph structure, but existing LLM–GNN methods often depend on expensive textual augmentation, external APIs, or a single fine-tuned encoder whose representations may be brittle for downstream graph learning. This study addresses how to obtain diverse, high-quality text-derived node representations and combine them effectively for node classification without augmenting node text. We introduce DIVERGE, which repeatedly fine-tunes one LLM on native node texts and labels using diverse LoRA initializations, trains a GNN on each resulting embedding set, and adaptively ensembles the GNN pool with trainable model- and class-level weights. On Cora, CiteSeer, PubMed, WikiCS, and OGBN-ArXiv, DIVERGE consistently outperformed classical GNNs and hybrid LLM–GNN baselines, reaching the best reported accuracy and Macro-F1 on all five benchmarks. Different initializations produced complementary embeddings, and learned ensembling outperformed uniform averaging. These findings show that representation diversity induced from native graph text can match or exceed augmentation-based approaches while reducing computational and API cost. Because all encoding runs on locally fine-tuned models without transmitting node text to third-party services, DIVERGE is also well suited to privacy-sensitive settings such as institutional repositories, enterprise document networks, and regulated biomedical corpora.

1. Introduction

Text-attributed graphs (TAGs) model entities and their relationships through both network structure and associated text, appearing in social networks, citation graphs, and e-commerce platforms (Zhang et al., 2025a; He et al., 2024). **Graph neural networks (GNNs)** have traditionally processed these graphs using shallow embeddings or fine-tuned language models such as BERT (Devlin et al.). However, such encoders often fail to capture the semantic depth and world knowledge required for accurate node classification (He et al., 2024; Zhang et al., 2025a).

Large language models (LLMs), including GPT and LLaMA, offer stronger contextual understanding and have motivated a new generation of LLM–GNN methods (Brown et al., 2020; Touvron et al., 2023). Integrating LLMs with graphs nevertheless remains challenging because API-based annotation scales poorly to millions of nodes, exposes proprietary or sensitive node text to third-party cloud providers, and because LLM-generated features can introduce noise, hallucinations, and inference-time distribution shift (Jaiswal et al., 2024; Xie et al., 2025; Zhang et al., 2025a).

Recent research studies have addressed these hurdles through several innovative strategies:

- **Explanations as Features:** Approaches such as **TAPE** leverage the reasoning abilities of LLMs by using their decision-making explanations as vectorial node features, which significantly boosts GNN performance without requiring full LLM fine-tuning (He et al., 2024).
- **Efficient Integration and Inference:** Frameworks like **E-LLaGNN** utilize LLMs as an on-demand service to enhance only a tactical fraction of nodes, thereby reducing the computational footprint and facilitating **LLM-free inference** for industrial deployment (Jaiswal et al., 2024).
- **Topological Structure Refinement:** Models like **GraphEdit** and **GAGA** employ LLMs to learn complex node relationships globally, effectively denoising connections by removing unreliable edges and adding reliable ones based on semantic similarity (Guo et al., 2025; Hu et al., 2024; Sun et al., 2026).
- **Cost-Effective Labeling:** New active self-training frameworks, such as **Locle**, identify "critical" samples for LLM annotation while utilizing GNN-based graph rewiring to refine labels, achieving state-of-the-art results for **label-free node classification** at a minimal cost (Zhang et al., 2025a).

ORCID(s):

Despite this progress, three limitations motivate our work. First, augmentation-heavy pipelines such as TAPE (He et al., 2024) depend on external LLM APIs and can hurt accuracy when node texts are already informative (Section 4). Second, sending full node corpora to remote APIs creates privacy, retention, and compliance risks in institutional, enterprise, and regulated domains where graph text must remain on premises. Third, existing ensemble variants typically average GNN logits uniformly, which cannot exploit class-specific strengths of different representations. We therefore propose **DIVERGE**, a framework that induces diverse semantic views through multiple LoRA initializations and combines the resulting GNNs with adaptive, trainable weighting—without textual augmentation, paid API calls, or off-site disclosure of node text.

From an **information-processing perspective**, TAG node classification supports core tasks in bibliometrics, digital libraries, knowledge-base curation, knowledge graph and recommendation over linked documents (Zhang et al., 2024; Ramírez Molina et al., 2026; Wang et al., 2026). Citation and hyperlink graphs encode how scientific ideas, web pages, and catalogued entities relate to one another; accurate categorization of nodes directly affects literature discovery, expert finding, and knowledge-network maintenance.

Our main contributions are as follows:

- We propose **DIVERGE**, a novel framework for text-attributed graph learning that generates diverse semantic representations using multiple parameter-efficient LLM fine-tuning processes with different LoRA initialization strategies, without requiring textual augmentation or external LLM API calls.
- We introduce an adaptive ensemble learning framework that combines multiple GNNs trained on heterogeneous textual representations using trainable weighting mechanisms, including model-level and class-level aggregation strategies.
- We demonstrate that diverse representations can be effectively induced solely from the original graph texts and graph structure, eliminating the need for costly LLM-generated augmented texts while reducing computational and financial overhead and keeping all node text within the operator's trusted infrastructure—an important requirement for privacy-sensitive and compliance-constrained deployments.
- We highlight a practical advantage of local, API-free encoding: DIVERGE never transmits proprietary node descriptions to external LLM services, which supports on-premises deployment in bibliographic systems, enterprise knowledge graphs, and regulated biomedical corpora where data residency and confidentiality constraints prohibit cloud-based augmentation.
- Extensive experiments on benchmark datasets, including Cora, CiteSeer, PubMed, WikiCS, and OGBN-ArXiv, show that the proposed method consistently outperforms existing GNN and hybrid LLM-GNN baselines across Accuracy and Macro-F1 metrics.
- Through ablation studies and statistical analysis, we show that different LoRA initialization strategies produce complementary semantic representations and that trainable ensemble learning significantly improves classification performance compared to simple averaging methods.

The structure of the paper is as follows: in Section 3, we review existing studies on integrating LLMs and GNNs for text-attributed graphs. Section 4 presents the motivation and preliminaries of the proposed framework. In Section 5, the proposed DIVERGE methodology is described in detail, including the diverse LoRA initialization strategies, representation extraction process, and adaptive ensemble learning mechanisms. Section 6 introduces the datasets, baselines, implementation details, and evaluation metrics used in the experiments. Section 7 reports the experimental results and ablation studies. Section 8 interprets these findings, articulates their theoretical and practical implications, and positions DIVERGE relative to prior work. Section 9 discusses the limitations of the proposed approach, and Section 10 concludes the paper and outlines directions for future work.

2. Research Objectives

Motivated by the limitations of augmentation-heavy LLM–GNN pipelines, this study addresses the following objectives:

1. To induce diverse, complementary node representations from native graph texts using multiple LoRA initialization strategies, without textual augmentation, paid LLM APIs, or off-site exposure of sensitive node text.

2. To design adaptive ensemble mechanisms that weight heterogeneous GNN outputs at model and class levels.
3. To evaluate DIVERGE against GNN, encoder-, enhancer-, and predictor-based LLM–GNN baselines on seven TAG benchmarks from five domains, with three LLM encoders and three GNN encoders.
4. To analyze when diversity-driven encoding outperforms augmentation-based approaches and quantify gains statistically.

3. Related Works

LLMs have recently been integrated with GNNs for processing TAGs. Based on existing studies, We categorize the role of LLMs into three main paradigms: Encoder, Enhancer, and Predictor (Wu et al., 2025; Jin et al., 2024; Chen et al., 2024b; Li et al., 2024a).

3.1. LLMs as Encoders

The **Encoder** paradigm uses LLMs to map node text to vector representations that downstream GNNs consume (He et al., 2024; Huang et al., 2024; Zhu et al., 2024). Early pipelines relied on shallow features (bag-of-words, skip-gram); recent methods employ pre-trained LMs or LLMs as text encoders. **ENGINE** (Zhu et al., 2024) optimizes end-to-end efficiency for large TAGs through parameter-efficient LLM tuning. **GraphAdapter** (Huang et al., 2024) injects structural signals into frozen LLMs via lightweight adapters (Houlsby et al., 2019). **WalkLM** (Tan et al., 2023) textualizes random walks to joint learning of text and topology, while **One-for-All (OFA)** (Liu et al., 2024) seeks a unified semantic space across graph domains. **LEADING** (Xue et al., 2025) further reduces label demand through neighbor decoupling during end-to-end LM–GNN training. **STAGE** (Zolnai-Lucas et al., 2024) demonstrates that frozen LLM embeddings combined with ensemble GNN training can be competitive on academic TAGs with minimal pipeline complexity.

Wu et al. (2025) release **LLMNodeBed** and systematically ask *when* LLMs help node classification. Their analysis shows that (a) encoder-style LLM features consistently outperform bag-of-words and shallow LM baselines on most TAG benchmarks, (b) enhancement-heavy pipelines do not always justify their cost, and (c) predictor-style LLM methods are sensitive to prompting and scale. These findings motivate DIVERGE's focus on strong *local* encoders rather than paid API augmentation.**LLM-BP** (Wang et al., 2025a) proposes task-adaptive LLM embeddings and belief-propagation-style aggregation, improving cross-graph generalization on 11 TAG datasets. DIVERGE differs from LLM-BP by inducing diversity through multiple LoRA initializations of a *single* backbone rather than task-adaptive prompting, and by learning ensemble weights over GNN logits rather than propagating beliefs on a single embedding space.

3.2. LLMs as Enhancers

Enhancer methods modify node text, graph structure, or labels before or during GNN training. **TAPE** (He et al., 2024) generates LLM explanations and zero-shot predictions to enrich node text, then trains multiple GNNs whose logits are *uniformly averaged*. Although effective on several benchmarks, TAPE requires external LLM API access and can introduce semantic noise when native texts are already informative (Section 4). Structural enhancers include **RoSE** (Sun et al., 2026) and **GraphEdit** (Guo et al., 2025). **LLM4NG** (Yu et al., 2025) synthesizes nodes and edges for few-shot TAG learning.

A second enhancer thread targets **cost and label efficiency**. **E-LLaGNN** (Jaiswal et al., 2024) enhances only critical nodes to enable LLM-free inference at deployment. **Locle** (Zhang et al., 2025a) combines active LLM labeling with GNN-based graph rewiring for label-scarce node classification. **GAGA** (Xie et al., 2025) annotates representative nodes and aligns an annotation graph with the TAG, requiring as little as 1% labeled data in reported experiments. **UltraTAG** (Zhang et al., 2026) modularizes augmentation, text encoding, and training into a unified robust pipeline (with extensions such as UltraTAG-S for sparse text/edges). **SKETCH** (Zhou et al., 2025) decouples semantic and structural aggregation: it retrieves related node texts and distant structural contexts before long-context LM training, reducing reliance on conventional GNN propagation while improving efficiency.

Collectively, enhancer methods demonstrate that LLMs can improve TAG learning by enriching inputs—but often at the cost of API fees, hallucination risk, or complex multi-stage pipelines. DIVERGE deliberately avoids textual augmentation and instead asks whether comparable gains can arise from *diverse encodings of native text*.

3.3. LLMs as Predictor

In the **Predictor** paradigm, LLMs directly perform reasoning tasks on text-attributed graphs. Methods like **GRAPHTEXT** transform graph structures into "Graph-Syntax Trees" that are flattened into textual sequences, allowing

Table 1
Comparison of DIVERGE with representative recent TAG methods (2024–2025).

Method	Augment. text?	External API?	Diversity source	Aggregation
TAPE (He et al., 2024)	Yes	Yes	Multi-feature/GNN pool	Uniform average
UltraTAG (Zhang et al., 2026)	Yes (module)	Optional	Pipeline modules	Task-specific
LLM-BP (Wang et al., 2025a)	No	Optional	Task-adaptive encoder	Belief propagation
SKETCH (Zhou et al., 2025)	No (retrieval)	No	Retrieved contexts	LM-centric
STAGE (Zolnai-Lucas et al., 2024)	No	No	Frozen LLM	Ensemble GNN
DIVERGE (ours)	**No**	**No**	**Multi-init LoRA**	**Learned ensemble**

LLMs to reason over the graph as a text-generation task (Zhao et al., 2023). Other approaches adapt LLMs to better capture structural knowledge by mapping graphs into ordered node sequences or injecting structural embeddings into the LLM's input space using projectors or prefix tuning, as seen in **DGTL** (Qin et al., 2024) and **LLaGA** Chen et al. (2024a). Additionally, the **ZeroG** framework explores cross-dataset zero-shot transferability, leveraging a shared semantic space to apply graph knowledge without task-specific fine-tuning (Li et al., 2024b).These methods lie outside our encoder–GNN design and are included only as baselines in Section 6.

3.4. Ensemble learning and representation diversity

Ensembling multiple graph learners is not new, but *how* models are combined varies widely. TAPE averages GNN logits with fixed equal weights (He et al., 2024). STAGE and LLMNodeBed benchmarks show that even simple encoder ensembles can be strong (Zolnai-Lucas et al., 2024; Wu et al., 2025). LLM-BP advances aggregation through learned, graph-adaptive belief propagation (Wang et al., 2025a). Yet most LLM–GNN pipelines still derive diversity from *different modalities or pipeline stages* (e.g., augmented vs. original text, LM vs. GNN), not from multiple parameter-efficient trajectories of the same encoder.

In parallel, parameter-efficient fine-tuning (PEFT) research shows that initialization and low-rank adaptation strategy affect the semantic subspace an LM converges to (Hu et al., 2021; Meng et al., 2024; Paischer et al., 2025; Li et al., 2023; Zhang et al., 2025b; Wang et al., 2025b). DIVERGE extends this insight to TAGs: we treat **LoRA initialization diversity** as a controllable source of complementary node representations and combine the resulting GNNs with **learned** model- and class-level weights rather than uniform averaging.

3.5. Research gap and position of DIVERGE

Table 1 summarizes how DIVERGE relates to representative recent methods. No prior TAG approach simultaneously (1) induces representation diversity through **multi-initialization LoRA** on native node text, (2) trains a **pool of GNNs** on complementary embeddings, and (3) fuses predictions with **learned ensemble weights**—all **without** LLM-based textual augmentation or external API calls. Methods such as TAPE and UltraTAG rely on augmentation; LLM-BP and SKETCH introduce diversity through task-adaptive prompting or retrieval;STAGE use a single encoding trajectory. DIVERGE fills this gap by showing that initialization-driven diversity plus adaptive ensembling can match or exceed augmentation-based performance (Section 8.1) while remaining deployable in privacy-sensitive information systems (Section 8).

4. Preliminaries

4.1. Motivation

Recent studies on TAGs have shown that the quality and richness of node-associated texts play a critical role in the final performance of graph learning models (He et al., 2024). In citation networks and scientific document graphs, node texts often already contain highly informative semantic content that can strongly support downstream node classification tasks. Consequently, the effectiveness of graph representation learning increasingly depends on how efficiently this semantic information is extracted and utilized.

LLMs have recently emerged as powerful semantic encoders due to their strong contextual understanding and extensive linguistic knowledge. Several recent methods utilize LLMs to augment node texts by generating explanations, summaries, or pseudo-label descriptions for graph nodes (He et al., 2024). The underlying assumption of these approaches is that additional generated textual information can enrich node semantics and improve graph learning performance.

Table 2
Accuracy of GNNs using different initial node features on the Cora dataset.

Type of Initial Node Features	Accuracy (%)
Embeddings of original graph node texts	89.14
Embeddings of texts augmented by LLMs (zero-shot prediction)	87.77
Bag-of-words features	85.34

However, experimental observations reveal that textual augmentation is not always beneficial, particularly in graphs whose original node texts are already rich and informative. In such scenarios, generated texts may introduce semantic noise, hallucinated information, or incorrect predictions, which can ultimately degrade downstream graph performance. Table 2 demonstrates this phenomenon on the Cora dataset, where embeddings extracted directly from the original graph texts outperform embeddings derived from LLM-augmented texts.

Furthermore, augmentation-based methods impose substantial computational and financial costs. Many existing approaches rely on repeated calls to closed-source LLM APIs in order to generate augmented texts for all graph nodes. According to He et al. (2024), augmenting ogbn-arxiv (169,343 nodes) with GPT-3.5 costs about 128 US dollars; with GPT-4o the same pass costs about 400 US dollars (Section 7.8.5). Such costs become increasingly prohibitive for large-scale graphs and real-world applications. Equally important, API-based augmentation requires uploading node texts—which may include unpublished manuscripts, internal reports, patient-related abstracts, or other confidential material—to third-party providers, raising data-residency, retention, and regulatory concerns. DIVERGE avoids this risk by fine-tuning and encoding entirely on local graph text. These observations raise a more fundamental question: Can competitive or even superior performance be achieved without performing any textual augmentation, solely by making more effective use of the original graph texts and graph structure?

In this work, we hypothesize that representation diversity itself can serve as an alternative source of complementary information for text-attributed graph learning. Instead of generating additional texts, we investigate whether multiple semantic views can be extracted directly from the original node texts through different parameter-efficient fine-tuning trajectories of the same LLM.

Recent studies in optimization and representation learning suggest that different parameter initializations may guide neural networks toward different local minima, potentially producing complementary semantic representations. Motivated by this observation, we employ multiple LoRA fine-tuning processes using different initialization strategies. Although all models are trained on identical graph texts and labels, each initialization strategy can induce a distinct semantic interpretation of node texts, resulting in diverse node-level embeddings.

These diverse representations are subsequently used to train multiple independent GNNs. However, simply combining the outputs of these graph models through uniform averaging may not fully exploit the complementary strengths of different representations. In practice, some representations may perform better for specific classes or graph regions, while others may capture alternative semantic structures more effectively.

Therefore, this work additionally investigates adaptive ensemble learning strategies capable of learning the relative importance of different graph models. By assigning trainable weights at both the model and class levels, the proposed framework aims to more effectively combine complementary representations and improve overall graph learning performance.

Based on these motivations, we propose **DIVERGE**, a framework that integrates diverse parameter-efficient LLM fine-tuning with adaptive ensemble learning for robust and scalable learning on text-attributed graphs.

4.2. Main Objective of the Proposed Method

The main objective is to generate diverse representations of node texts (without augmenting or modifying the original node texts) and subsequently apply ensemble learning on GNNs trained using these representations.

Graphs with textual features are graphs in which each node is associated with a text (document). In this work, a graph with textual features is defined as follows:

$$G = (V, E, D, Y), \tag{1}$$

where:

- $V = \{v_1, \ldots, v_N\}$ is the set of nodes,

- $E \subseteq V \times V$ is the set of edges,
- $D = \{d_{v_1}, \ldots, d_{v_N}\}$ is the set of node texts, where d_{v_i} is the text corresponding to node v_i,
- $Y = \{y_1, \ldots, y_N\}$ is the set of node labels, and
- $y_i \in \{1, \ldots, C\}$ denotes the label of node v_i among C classes.

Assume that a pre-trained LLM is available as a parametric function $f(\cdot; \mathbf{W})$. The input of this function is the node text, and its parameters correspond to the pre-trained weights:

$$f(x; \mathbf{W}), \qquad \mathbf{W} \in \mathbb{R}^{d \times k}, \tag{2}$$

where x is the input text and $\mathbf{W}$ represents the model weights.

For fine-tuning the language model, the training set consisting of (text, label) pairs is defined as:

$$(D_{\text{train}}, Y_{\text{train}}) = \{(d_i, y_i)\}_{i \in \mathcal{I}_{\text{train}}}. \tag{3}$$

The objective is to minimize the loss function over the training data:

$$\min_{\mathbf{W}} \mathcal{L}\big(f(x; \mathbf{W}), y\big). \tag{4}$$

However, optimizing all parameters $\mathbf{W}$ (with dimensions $d \times k$) is computationally expensive; therefore, parameter-efficient fine-tuning methods are employed. In this work, the selected method is LoRA, which learns only a low-rank update instead of training all parameters:

$$\mathbf{W}' = \mathbf{W} + \Delta\mathbf{W}, \qquad \Delta\mathbf{W} = \mathbf{BA}, \tag{5}$$

where:

$$\mathbf{B} \in \mathbb{R}^{d \times r}, \qquad \mathbf{A} \in \mathbb{R}^{r \times k}, \qquad r \ll \min(d, k). \tag{6}$$

As a result, the optimization problem is reduced to training only the parameters $\mathbf{A}, \mathbf{B}$:

$$\min_{\mathbf{A},\mathbf{B}} \mathcal{L}\big(f(x; \mathbf{W} + \mathbf{BA}), y\big). \tag{7}$$

After optimization, the resulting model is denoted by $\bar{f}$, which combines the original weights and the low-rank update:

$$\bar{f}(x) = f(x; \mathbf{W} + \mathbf{BA}). \tag{8}$$

For initializing the low-rank matrices, an initialization strategy is employed:

$$(\mathbf{A}^{(0)}, \mathbf{B}^{(0)}) \sim \mathcal{I}. \tag{9}$$

5. Methodology

Algorithm 1 Proposed DIVERGE Framework

Require: Graph $G = (V, E, D, Y)$, pre-trained LLM f, number of LoRA runs M
Ensure: Predicted node labels $\hat{Y}$
  for $i = 1$ to M **do**
    Initialize LoRA parameters with initialization $\mathcal{I}_i$
    Fine-tune LLM to obtain adapted model $\bar{f}_i$
    Extract node features $\mathbf{Z}^{(i)}$ from node texts using $\bar{f}_i$
    Train GNN on $(G, \mathbf{Z}^{(i)})$
    Obtain node logits $\mathbf{L}^{(i)}$
  end for
  Collect logits $\bar{\mathbf{L}} = \{\mathbf{L}^{(i)}\}_{i=1}^{M}$
  Train ensemble model on $\bar{\mathbf{L}}$
  Aggregate logits and predict labels $\hat{Y}$
  return $\hat{Y}$

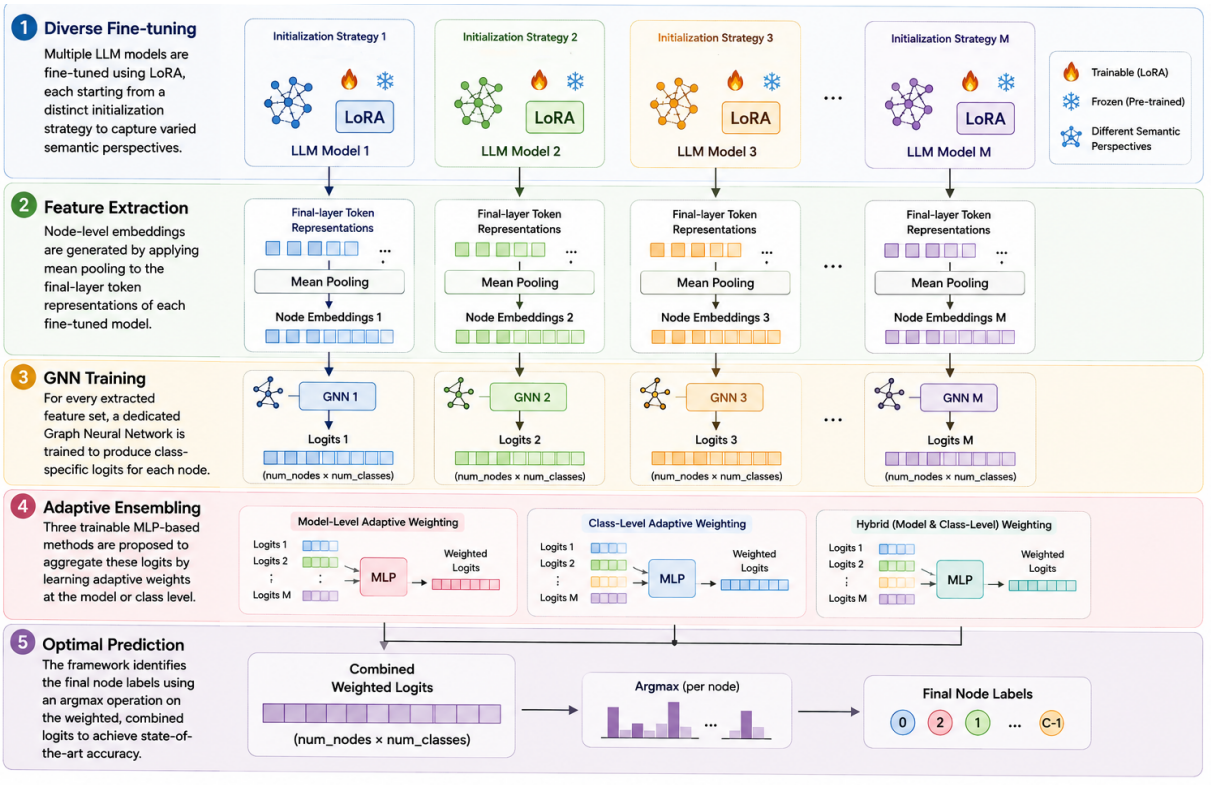


Figure 1: Overview of a diverse fine-tuning and ensemble framework combining multiple LoRA-adapted LLMs models, GNN-based node representation learning, and adaptive MLP fusion for final node classification.

5.1. Training LoRA Parameters of LLMs with Different Initializations

To generate diverse representations, multiple LoRA fine-tuning runs with different initializations are performed. In other words, for each iteration i, an independent initialization strategy $\mathcal{I}_i$ is defined:

$$(\mathbf{A}_i^{(0)}, \mathbf{B}_i^{(0)}) \sim \mathcal{I}_i, \qquad i = 1, \ldots, M. \tag{10}$$

Then, for each initialization, the language model is fine-tuned only through training the low-rank parameters:

$$\min_{\mathbf{A}_i, \mathbf{B}_i} \mathcal{L}\big(f(x; \mathbf{W} + \mathbf{B}_i\mathbf{A}_i), y\big). \tag{11}$$

By solving the above optimization problem, the i-th fine-tuned model is obtained as:

$$\bar{f}_i(x) = f(x; \mathbf{W} + \mathbf{B}_i\mathbf{A}_i). \tag{12}$$

Finally, the collection of fine-tuned language models is defined as:

$$\mathcal{M} = \{\bar{f}_i\}_{i=1}^{M}. \tag{13}$$

5.2. Extracting Node-Level Feature Sets Using the Trained Models

At this stage, the goal is to convert the text of each node into a node-level feature vector using the models $\bar{f}_i$ (Equation 12).

For node v, let the corresponding text be denoted by d_v, which after tokenization consists of T tokens:

$$d_v = \{x_1, \ldots, x_T\}. \tag{14}$$

Since node texts are processed in mini-batches, each sequence is padded (or truncated) to a common length T, and the tokenizer returns an attention mask $m_{v,t} \in \{0, 1\}$, where $m_{v,t} = 1$ if x_t is a valid token of d_v and $m_{v,t} = 0$ if it is a padding token.

The output of the final layer of model $\bar{f}_i$ for the tokens is considered as a sequence of vectors:

$$\mathbf{H}_v^{(i)} = \left(\mathbf{h}_1^{(i)}, \ldots, \mathbf{h}_T^{(i)}\right), \qquad \mathbf{h}_t^{(i)} \in \mathbb{R}^p, \tag{15}$$

where $\mathbf{h}_t^{(i)}$ is the representation of token t in the final layer.

To construct a node-level representation, mean pooling over the valid (non-padding) tokens is used:

$$\mathbf{z}_v^{(i)} = \frac{\sum_{t=1}^{T} m_{v,t}\, \mathbf{h}_t^{(i)}}{\sum_{t=1}^{T} m_{v,t}}. \tag{16}$$

Hence, padded positions are excluded from both the numerator and the denominator, and the representation of node v is the average of the final-layer embeddings of its actual text tokens only; it is therefore independent of the padding length used in the batch.

By stacking node vectors together, the node-level feature matrix for the i-th model is obtained:

$$\mathbf{Z}^{(i)} = \left[\mathbf{z}_{v_1}^{(i)}, \ldots, \mathbf{z}_{v_N}^{(i)}\right]^{\top} \in \mathbb{R}^{N \times p}. \tag{17}$$

Consequently, the set of extracted features from all fine-tuned language models is:

$$\mathcal{F} = \{\mathbf{Z}^{(i)}\}_{i=1}^{M}. \tag{18}$$

5.3. Training Multiple GNNs Using the Extracted Features

After obtaining the feature sets, a GNN is trained for each feature matrix $\mathbf{Z}^{(i)}$. The GNN is defined as a function $g(\cdot)$ operating on the graph and node features:

$$g_i = \mathrm{GNN}(G, \mathbf{Z}^{(i)}), \qquad i = 1, \ldots, M. \tag{19}$$

For each node v, the output of network g_i is a logit vector over the C classes:

$$\boldsymbol{\ell}_v^{(i)} = \left(\ell_{v,1}^{(i)}, \ldots, \ell_{v,C}^{(i)}\right) \in \mathbb{R}^C. \tag{20}$$

The logits of all nodes for the i-th model are summarized as:

$$\mathbf{L}^{(i)} = \{\boldsymbol{\ell}_{v_1}^{(i)}, \ldots, \boldsymbol{\ell}_{v_N}^{(i)}\}. \tag{21}$$

Finally, the set of outputs (logits) from all GNNs is:

$$\bar{\mathbf{L}} = \{\mathbf{L}^{(i)}\}_{i=1}^{M}. \tag{22}$$

5.4. Training the Ensemble Learning Model

At this stage, an ensemble learning framework is designed using the logits obtained from the M GNNs. Unlike methods such as TAPE, which employ simple averaging of logits, the proposed method assumes that not all models contribute equally; therefore, adaptive weighting can improve the final performance.

The data used for training the ensemble learning model is defined as:

$$(\bar{\mathbf{L}}_{\text{train}}, y_{\text{train}}). \tag{23}$$

All stages of DIVERGE (LoRA fine-tuning, GNN training and ensemble training) use the same train/validation/test split, generated with a single fixed random seed. The ensemble head is trained only on the logits of the training nodes; the validation nodes are used to select the training epoch and the hyperparameters of the head, and the test nodes are used only once, for the final evaluation. In all three methods below, the MLP takes the logits of each node as input, so the learned weights are node-conditioned rather than global constants.

Method 1: Model-Level Weighting In this method, each graph model is assigned a learnable weight in order to determine the relative importance of each model. For node v, the weight vector is produced by an MLP with parameters ϕ from the concatenated logits of the M GNNs:

$$\boldsymbol{\theta}_v = \mathrm{softmax}\big(\mathrm{MLP}_\phi([\boldsymbol{\ell}_v^{(1)} \,\|\, \dots \,\|\, \boldsymbol{\ell}_v^{(M)}])\big) \in \mathbb{R}^M, \qquad \sum_{i=1}^{M} \theta_{v,i} = 1, \;\; \theta_{v,i} \geq 0. \tag{24}$$

The softmax layer at the output of the MLP satisfies the above constraints. The combined logits for node v are computed as:

$$\bar{\boldsymbol{\ell}}_v = \sum_{i=1}^{M} \theta_{v,i}\, \boldsymbol{\ell}_v^{(i)}. \tag{25}$$

The parameters of the ensemble model are trained by minimizing the cross-entropy loss over the training nodes:

$$\min_{\phi} \sum_{v \in \mathcal{V}_{\text{train}}} \mathcal{L}\big(\bar{\boldsymbol{\ell}}_v, y_v\big). \tag{26}$$

Method 2: Class-Level Weighting In this method, an independent weight is assigned to each model and each class, allowing each model to contribute more strongly to the classes in which it performs better:

$$\boldsymbol{\Theta}_v \in \mathbb{R}^{M \times C}, \qquad \sum_{i=1}^{M} \Theta_{v,i,c} = 1, \;\; \Theta_{v,i,c} \geq 0 \;\; \forall c. \tag{27}$$

The combined logits for node v are computed as:

$$\bar{\ell}_{v,c} = \sum_{i=1}^{M} \Theta_{v,i,c}\, \ell_{v,c}^{(i)}, \qquad c = 1, \dots, C. \tag{28}$$

The weights $\boldsymbol{\Theta}_v$ are produced by an MLP that takes the same concatenated logits as Method 1 as input, with a softmax layer applied separately for each class.

Method 3: Class-Level Weighting with Independent Networks This method is similar to Method 2, with the difference that an independent MLP is defined for each class c. The goal of this design is to increase flexibility in learning complex and distinct weighting patterns across classes:

$$\boldsymbol{\Theta}_{v,:,c} = \mathrm{softmax}\big(\mathrm{MLP}_c([\ell_{v,c}^{(1)}, \dots, \ell_{v,c}^{(M)}])\big), \qquad c = 1, \dots, C, \tag{29}$$

where the MLP of class c receives only the class-c logits of the M GNNs for node v.

Table 3
Specifications of the text-attributed graph benchmarks used in this study. Train/validation/test counts follow a random 60/20/20 split. Average degree is computed as $2|E|/|V|$ for undirected graphs. Average tokens are measured with the LLaMA-3.2 tokenizer before truncation.

Statistic	Cora	CiteSeer	PubMed	OGBN-ArXiv	WikiCS	Reddit	Computers
Nodes	2,708	3,186	19,717	169,343	11,701	33,434	87,229
Edges	5,429	4,277	44,338	1,166,243	215,863	134,236	628,274
Classes	7	6	3	40	10	2	10
Train nodes	1,624	1,912	11,830	101,606	7,020	20,060	52,337
Validation nodes	541	637	3943	33869	2340	6,686	17,445
Test nodes	542	638	3944	33869	2341	6,688	17,447
Avg. tokens per node	165	189	369	217	530	175	112
Avg. degree	3.89	2.65	4.49	13.67	36.89	8.03	14.41
Node text	Title + abstract	Title + abstract	Title + abstract	Title + abstract	Full article	User posts	Product review
Domain	Machine learning	Computer science	Biomedical	Computer science	Computer science	Social media	E-commerce

Final Prediction After computing the aggregated logits for each node, the predicted label is determined using the argmax operation:

$$\hat{y}_v = \arg\max_{c\in\{1,\dots,C\}} \bar{\ell}_{v,c}. \tag{30}$$

6. Evaluation

6.1. Datasets

We evaluated DIVERGE on seven text-attributed graph (TAG) benchmarks obtained from the **LLMNodeBed** repository ([dataset] Wu et al., 2025; Wu et al., 2025), which provides standardized node texts, graph structures, and labels for fair comparison of LLM–GNN methods. The original dataset sources are Cora (McCallum et al., 2000), CiteSeer (Giles et al., 1998), PubMed (Sen et al., 2008), OGBN-ArXiv (Hu et al., 2020), WikiCS (Mernyei and Cangea, 2022), Reddit (Huang et al., 2024), and Computers (**?**). Besides the five citation and hyperlink graphs, Reddit (social media) and Computers (e-commerce) test DIVERGE on two further domains whose texts are informal user-generated posts and reviews rather than scientific or encyclopedic writing. Table 3 summarizes graph scale, textual properties, and split sizes; together, the seven benchmarks comprise **327,318 nodes** and **2.20 million edges**.

Source and license. Cora, CiteSeer, and PubMed are classic citation-network benchmarks released for research use in the graph-learning literature (McCallum et al., 2000; Giles et al., 1998; Sen et al., 2008). OGBN-ArXiv is distributed under the Open Graph Benchmark (OGB) license (Hu et al., 2020). WikiCS is publicly available for non-commercial research (Mernyei and Cangea, 2022). Reddit and Computers are public research releases of the TAG benchmarks of Huang et al. (2024) and **?**. In our experiments, all graphs and node texts were accessed through the LLMNodeBed Hugging Face release ([dataset] Wu et al., 2025), which harmonizes raw texts and graph topology across TAG baselines.

Node text content and label definition. Each node represents a document, a user, or a product associated with a categorical label:

- **Cora:** Each node is a machine-learning publication. Node text concatenates the paper **title and abstract** (mean 165 tokens). Labels denote one of seven research topics (e.g., neural networks, rule learning, reinforcement learning).
- **CiteSeer:** Each node is a computer-science publication with **title and abstract** text (mean 189 tokens). Labels indicate one of six paper categories.
- **PubMed:** Each node is a biomedical abstract related to diabetes. Node text consists of the **title and abstract** (mean 369 tokens). Labels indicate one of three disease categories (experimental induced diabetes, type 1 diabetes, type 2 diabetes).

- **OGBN-ArXiv:** Each node is a computer-science paper indexed in the arXiv MAG corpus. Node text combines **title and abstract** (mean 217 tokens). Labels correspond to one of 40 arXiv subject areas (e.g., cs.AI, cs.LG).
- **WikiCS:** Each node is a Wikipedia article from the computer-science subset. Node text is the **full article content** (mean 530 tokens), making it the most text-dense benchmark in our study. Labels denote one of ten Wikipedia category tags.
- **Reddit (social media):** Each node is a Reddit user, and node text is the concatenation of the user's **most recent posts** (mean 175 tokens). The binary label indicates whether the user is popular or normal.
- **Computers (e-commerce):** Each node is a computer-related product from Amazon, and node text is a **user review** of the product (mean 112 tokens). Labels denote one of ten product categories.

Graph construction. For Cora, CiteSeer, and PubMed, undirected edges represent mutual citation links between papers. For OGBN-ArXiv, directed citation links from the OGB release were treated as **undirected** edges to match standard GNN-based TAG protocols (Wu et al., 2025). For WikiCS, undirected edges represent hyperlinks between Wikipedia pages. For Reddit, an edge connects two users who replied to each other; for Computers, an edge connects two products that are frequently co-purchased or co-viewed. **Self-loops were not added** to the adjacency matrices; graph convolution used the standard neighbor aggregation of each GNN encoder on the provided topology.

Data splits. All experiments use a **random 60/20/20 split** for training, validation, and test nodes, with split sizes reported in Table 3. Every experiment is repeated with four random seeds (42–45). Each seed draws its own 60/20/20 split, which is then held constant across all LoRA initializations, GNN models, and ensemble variants of that seed, so no stage sees the test nodes of its own split. Reported baseline results for classical GNNs and hybrid LLM–GNN methods are taken from Wu et al. (2025), which evaluates on the same LLMNodeBed benchmarks and split protocol; this ensures a like-for-like comparison. Where a baseline originally used a different split (e.g., the standard Planetoid per-class split), Wu et al. (2025) re-evaluated it under the unified TAG setting.

Text preprocessing. Node texts were tokenized with the tokenizer of each LLM encoder (LLaMA-3.2, SmolLM2 or Gemma-2). Sequences longer than **512 tokens** were truncated from the right; shorter sequences were padded during batching. Truncation affects 29% of the WikiCS nodes, 11% of PubMed, 5% of Reddit, and at most 1% of the other datasets. Empty or missing texts were not present in the LLMNodeBed releases used here; if truncation removed tail content (notably on WikiCS), mean pooling was applied over all non-padding tokens of the final LLM hidden layer. Bag-of-words baselines in comparative tables used the preprocessed token statistics bundled with LLMNodeBed (Wu et al., 2025).

Class distribution and imbalance. The ratio between the largest and the smallest class is 1.9 on PubMed, 2.9 on CiteSeer, 4.5 on Cora, and 9.1 on WikiCS. Reddit is exactly balanced (two classes of 16,717 users). Computers and OGBN-ArXiv have a long tail: the smallest Computers category has 120 products against 22,494 in the largest, and OGBN-ArXiv combines **40 fine-grained labels** with a largest-to-smallest ratio above 900. Because of this imbalance we report Macro-F1 alongside accuracy.

Summary. These benchmarks span citation and hyperlink graphs from machine learning, general computer science, biomedicine, and Wikipedia, a social network, and a product co-purchase graph, with node counts from 2.7K to 169K and average degrees from 2.65 to 36.89. This diversity allows us to test whether representation diversity and adaptive ensembling generalize across graph scale, text length, domain, and label granularity.

6.2. Baselines

The baseline methods used for comparison with the proposed method are categorized into four general groups:

- Classical GNN-based methods
- LLMs as Encoders
- LLMs as Enhancers
- LLMs as Predictors

In the first category, three well-known GNN methods were utilized: GCN (Kipf and Welling, 2017), GraphSAGE (Hamilton et al., 2017), and GAT (Veličković et al., 2018). In these methods, textual features were converted into numerical vectors using the **Bag-of-Words** approach. In other words, **shallow embeddings** were used to represent the textual information of the nodes.

In the second category, serving as encoders and without fine-tuning, two language models—Sentence-BERT (Reimers and Gurevych, 2019) and RoBERTa (Zhuang et al., 2021)—were employed to transform textual features into richer vector representations. These representations were then provided as input to the GNNs, and the models were trained in a hybrid fashion. Additionally, the ENGINE (Zhu et al., 2024) method was considered in the encoder role, which aims to achieve the best possible performance while optimizing training time and memory consumption, together with LLM-BP (Wang et al., 2025a), which combines a task-adaptive LLM encoder with belief propagation over the graph, and STAGE (Zolnai-Lucas et al., 2024), which ensembles GNNs trained on node embeddings from a single frozen LLM.

In the third category, LLMs were examined as enhancers. To this end, TAPE (He et al., 2024), which also inspired the proposed method, was used alongside E-LLaGNN (Jaiswal et al., 2024), which enriches a selected subset of nodes with LLM-generated text; UltraTAG (Zhang et al., 2026), which augments both node text and graph structure through a modular pipeline; and SKETCH (Zhou et al., 2025), which enhances node representations with retrieved textual and structural context.

Finally, in the fourth category acting as predictors, two methods—GraphGPT (Tang et al., 2024) and LLaGA (Chen et al., 2024a)—were reviewed, in which LLMs are used directly to perform the prediction task.

6.3. Experiments Settings and Implementation Details

LLM encoders. To transform node text into semantic vectors, we use three decoder LLMs with fewer than two billion parameters: LLaMA-3.2-1B (**?**), SmolLM2-1.7B (**?**) and Gemma-2-2B (**?**). All three are fine-tuned with LoRA (Hu et al., 2021) for sequence classification; SmolLM2 and Gemma-2 are loaded in 4-bit precision (QLoRA-style, **?**) so that a single run fits comfortably on one GPU.

GNN encoders. On each set of node embeddings we train three GNN encoders: GCN (Kipf and Welling, 2017), GIN (**?**) and GraphSAGE (Hamilton et al., 2017). Unless stated otherwise, the main results use LLaMA-3.2-1B with GraphSAGE; the other LLM and GNN encoders are compared in Sections 7.5 and 7.6.

Seeds and reporting. Every configuration is run with four random seeds (42–45). A seed fixes the 60/20/20 split, the LoRA and GNN initialization, and the data order; we report the mean $\pm$ standard deviation over the four seeds.

All experiments and implementations were performed on an RTX 5070 GPU. For data training in the **fully supervised** mode, the data split ratios for training, validation, and test sets were set to 60%, 20%, and 20%, respectively.

Hyperparameters. All hyperparameters are listed in Appendix A. The LLM and LoRA settings are shared by the three LLM encoders except for the loading precision and the caching batch size (Table 25). The GNN hyperparameters are selected separately for every LLM encoder, GNN encoder and dataset by a grid search of 512 configurations on validation accuracy (search space in Table 26); the selected values are given in Tables 27–29. The test set is never used for selection. We set $M = 5$ LoRA runs corresponding to the five initialization strategies below. The strategies selected for initializing the trainable LoRA parameters include Gaussian (Hu et al., 2021), Orthogonal (Zhang et al., 2025b), PiSSA (Meng et al., 2024), EVA (Paischer et al., 2025), and LoFTQ (Li et al., 2023):

- In the **Gaussian** method, parameters are initialized from a normal distribution.
- In the **Orthogonal** method, LoRA matrices are initialized orthogonally to maintain gradient stability.
- In the **PiSSA** method, initialization is performed using the Singular Value Decomposition (SVD) of pre-trained weights to align them with their dominant subspace.
- In the **EVA** method, by scaling singular values, only a controlled portion of the original weight variance is modeled at the start of training.
- Finally, in the **LoFTQ** method, LoRA parameters are initialized to approximate the quantization error residue, thereby compensating for accuracy loss caused by quantization.

To present the results of individual GNNs obtained through training on different representations, we use the name of the **initialization strategy** employed during the training of the LLM on the graph text:

$$\text{PiSSA} = \text{GNN}(G, Z_{\text{PiSSA}}), \tag{31}$$

$$\text{EVA} = \text{GNN}(G, Z_{\text{EVA}}), \tag{32}$$

$$\text{Orthogonal} = \text{GNN}(G, Z_{\text{Orthogonal}}), \tag{33}$$

$$\text{LoFTQ} = \text{GNN}(G, Z_{\text{LoFTQ}}), \tag{34}$$

$$\text{Gaussian} = \text{GNN}(G, Z_{\text{Gaussian}}). \tag{35}$$

In these relations, GNN represents the graph neural network, and G denotes the graph along with the textual features serving as input to the GNN. Furthermore, Z_{PiSSA} is the representation extracted by averaging the last layer output of a pre-trained LLM, whose LoRA parameters were initialized based on the PiSSA strategy and subsequently trained. Similarly, the other representations (Z_{EVA}, $Z_{\text{Orthogonal}}$, Z_{LoFTQ}, and Z_{Gaussian}) are obtained from models whose LoRA parameters were tuned and trained according to their corresponding initialization strategies.

6.4. Implementation Detail for Ensemble Learning

The ensemble heads are trained with Adam for 200 epochs on the logits of the training nodes. The number of MLP layers ($\{2, 3\}$), hidden size ($\{64, 128, 256\}$), dropout ($\{0.3, 0.5\}$) and learning rate ($\{10^{-4}, 10^{-3}, 10^{-2}\}$) are selected by grid search, and both the configuration and the training epoch are chosen by validation accuracy.

6.5. Evaluation Metrics

To evaluate the performance of the classification model, two metrics were employed: **Accuracy** and **Macro F1-score**. Each of these metrics highlights a different aspect of the model's predictive quality, providing a more comprehensive overview of the model's performance when considered together.

Accuracy is defined as the ratio of correctly classified samples to the total number of samples. While this metric provides a general overview of model performance, it may not be suitable for problems with imbalanced class distributions, as majority classes exert a disproportionate influence on the final value.

The **Macro F1-score** is obtained by calculating the F1-score for each class independently and then taking their unweighted arithmetic mean. In this metric, all classes are treated with equal importance; therefore, the model's performance on minority classes is well-reflected in the final result. Consequently, the Macro F1-score is an appropriate metric for assessing the uniformity of the model's performance across all classes.

7. Results

In Table 4 and Table 5, the performance of previous methods, including classical GNNs and hybrid LLM-GNN approaches, is reported alongside the proposed method in its three trainable ensemble versions, based on Accuracy and Macro-F1 metrics. The results of the predictor methods (GraphGPT and LLaGA) are taken from LLMNodeBed (Wu et al., 2025), whereas all other baselines were re-run by us under the same data splits and evaluation protocol as DIVERGE.

As observed, the proposed method achieves the best performance across all provided versions compared to previous baselines. These results demonstrate that the proposed framework has reached the **state-of-the-art** performance level.

Although the proposed method is not fully optimized regarding memory consumption, it generates rich and meaningful textual representations by relying on the original graph text. Ultimately, through the ensemble learning mechanism, it consistently improves model performance in fully supervised settings.

7.1. Comparison of Textual Embeddings Obtained from LLMs Using Different Initial Parameters

As described in the proposed method, an LLM is fine-tuned on node text data specifically for the graph text classification task. Upon completion of the training process, node-level representations are extracted by applying a mean pooling operator over the textual representations associated with each node.

To qualitatively compare these representations, the node embeddings generated by each model version were mapped into a two-dimensional space using the t-SNE method (Cai and Ma, 2022), the results of which are shown in Figure 2. As evident from the plots, each trained language model, despite using identical data and labels, has achieved distinct spatial patterns and node clusterings. These variations demonstrate that changes in the initialization of trainable parameters can lead to distinct semantic interpretations of node texts, consequently producing diverse node-level representations.

Table 4
Comparison of the accuracy of the proposed DIVERGE method with previous methods in the node classification task.

Method	Accuracy (%)						
	Cora	CiteSeer	PubMed	WikiCS	ArXiv	Computers	Reddit
GCN	87.32±1.42	74.52±1.67	89.24±0.23	82.59±0.31	70.92±0.18	82.01±0.41	63.25±0.27
SAGE	88.01±1.28	74.15±1.53	90.77±0.19	84.55±0.44	71.31±0.35	87.26±0.22	64.49±0.38
GAT	86.28±1.71	72.33±1.89	87.95±0.12	83.24±0.28	71.42±0.47	86.27±0.33	64.81±0.15
SenBERT-66M	80.52±1.15	73.16±1.34	93.23±0.08	86.91±0.21	72.24±0.39	77.48±0.46	60.73±0.29
RoBERTa-355M	83.86±1.56	74.89±1.22	93.63±0.37	87.13±0.14	70.22±0.26	78.99±0.09	62.13±0.43
ENGINE	85.10±1.83	73.22±1.47	90.42±0.42	85.13±0.36	73.29±0.11	87.63±0.25	65.30±0.48
TAPE	89.32±1.09	77.65±1.76	93.98±0.16	87.24±0.49	75.92±0.32	89.37±0.07	65.36±0.21
GraphGPT	82.29±0.26	74.67±1.15	93.54±0.22	82.54±0.23	75.15±0.14	86.78±1.14	60.72±1.47
LLaGA	87.55±1.15	76.73±1.70	90.28±0.91	84.03±1.10	74.49±0.23	90.41±0.12	**71.06**±0.38
E-LLaGNN	86.13±1.38	74.02±1.61	90.03±0.27	85.79±0.18	71.33±0.44	84.18±0.13	61.15±0.36
UltraTAG	87.22±1.24	76.69±1.92	90.59±0.31	85.93±0.45	72.84±0.09	84.76±0.38	62.22±0.17
LLM-BP	80.29±1.67	70.79±1.41	88.58±0.14	82.11±0.29	69.18±0.48	80.19±0.21	60.10±0.40
STAGE	88.90±1.13	76.44±1.58	93.49±0.46	86.73±0.08	74.27±0.24	88.23±0.34	64.92±0.12
SKETCH	82.30±1.79	71.51±1.26	90.01±0.22	84.25±0.41	72.44±0.16	83.18±0.07	62.05±0.33
DIVERGE Ensemble Method 1	**89.80**±1.39	**79.47**±0.97	**95.26**±0.14	**88.47**±0.22	**77.64**±0.31	**90.87**±0.00	64.80±0.00
DIVERGE Ensemble Method 2	**89.90**±1.32	**79.39**±0.85	**95.26**±0.19	**88.72**±0.17	**77.61**±0.24	**90.88**±0.00	65.59±0.00
DIVERGE Ensemble Method 3	**89.94**±1.15	**79.50**±0.81	**95.23**±0.11	**88.59**±0.28	**77.66**±0.18	**90.82**±0.00	65.68±0.00

Table 5
Comparison of the Macro-F1 performance of the proposed DIVERGE method with previous methods in the node classification task.

Method	Macro-F1 (%)						
	Cora	CiteSeer	PubMed	WikiCS	ArXiv	Computers	Reddit
GCN	86.18±1.51	70.95±1.88	88.61±0.34	81.41±0.12	49.85±0.29	79.10±0.18	63.05±0.45
SAGE	86.92±1.73	72.81±1.19	90.29±0.41	82.88±0.27	50.55±0.08	81.92±0.39	64.16±0.16
GAT	85.11±1.08	69.03±1.44	86.14±0.28	81.53±0.49	50.13±0.21	80.16±0.14	64.29±0.37
SenBERT-66M	77.29±1.92	71.32±1.63	92.82±0.17	83.81±0.35	51.27±0.43	73.63±0.11	57.83±0.08
RoBERTa-355M	81.19±1.31	71.98±1.85	92.97±0.09	85.13±0.22	55.39±0.47	74.88±0.33	60.28±0.19
ENGINE	83.57±1.64	69.99±1.12	89.21±0.25	82.74±0.48	56.66±0.36	83.05±0.17	63.23±0.07
TAPE	87.92±1.47	75.08±1.39	92.57±0.33	85.96±0.15	57.93±0.19	84.59±0.42	65.32±0.44
GraphGPT	74.08±4.36	61.04±1.24	80.98±11.22	73.92±0.61	56.80±0.08	74.19±1.75	39.97±1.77
LLaGA	84.97±3.97	72.59±1.70	90.00±0.80	82.37±1.73	58.08±0.29	86.97±0.34	62.14±15.59
E-LLaGNN	84.62±1.21	73.31±1.77	88.50±0.48	83.45±0.32	51.37±0.07	80.59±0.26	60.97±0.14
UltraTAG	85.47±1.86	74.56±1.08	88.93±0.13	83.66±0.19	53.73±0.38	80.73±0.44	62.14±0.29
LLM-BP	76.99±1.19	68.15±1.54	86.32±0.36	79.99±0.07	48.56±0.22	75.48±0.45	40.32±0.18
STAGE	86.88±1.58	74.78±1.93	91.19±0.21	84.72±0.43	56.36±0.39	84.31±0.16	64.65±0.28
SKETCH	82.05±1.35	70.14±1.48	88.48±0.44	81.94±0.26	51.23±0.31	79.53±0.19	61.79±0.09
DIVERGE Ensemble Method 1	**88.58**±1.60	**74.80**±0.77	**94.70**±0.16	**87.54**±0.27	**63.21**±0.34	**88.12**±0.00	64.75±0.00
DIVERGE Ensemble Method 2	**88.72**±1.51	**74.97**±0.82	**94.71**±0.21	**87.70**±0.19	**63.15**±0.29	**88.21**±0.00	**65.48**±0.00
DIVERGE Ensemble Method 3	**88.85**±1.37	**75.12**±0.83	**94.71**±0.13	**87.64**±0.23	**63.22**±0.41	**88.37**±0.00	**65.62**±0.00

7.2. Why do different LoRA initializations yield different representations?

The t-SNE plots in Figure 2 show that the five views differ, but they cannot tell whether the difference is caused by the initialization or by the randomness of training. We therefore follow the effect of the initialization through each stage of the pipeline: the adapter weights, the LLM embeddings, the GNN embeddings and the GNN predictions. We report the analysis for CiteSeer with Gemma-2-2B.

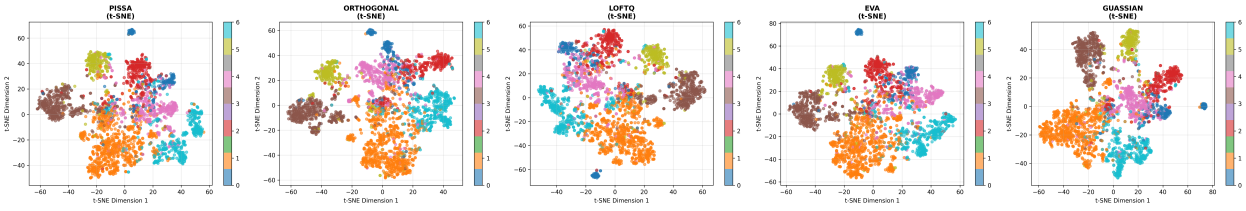


Figure 2: Overview of textual embeddings obtained from LLMs using different initial parameters.

Two-dimensional visualization of node embeddings using the t-SNE method for different versions of the language model trained on the Cora dataset. Differences in clustering patterns indicate the effect of parameter initialization on the semantic representations of nodes.

Mechanism. LoRA adds a rank-r update $\mathbf{BA}$ to a frozen weight $\mathbf{W} \in \mathbb{R}^{d_{out} \times d_{in}}$, so the adapter reads its input through the r-dimensional row space of $\mathbf{A}$ and writes into the column space of $\mathbf{B}$. When $\mathbf{B}$ starts at zero (Gaussian, Orthogonal and EVA), the first gradient of $\mathbf{B}$ is $\nabla_{\mathbf{B}}\mathcal{L} = \mathbf{g}(\mathbf{Ax})^\top$, so learning is confined to the input directions selected by the initial $\mathbf{A}$, and $\mathbf{A}$ itself moves little during fine-tuning (**?**). Each strategy chooses these directions differently: PiSSA takes the top-r singular vectors of $\mathbf{W}$ (Meng et al., 2024), LoftQ the singular vectors of the quantization residual (Li et al., 2023), EVA the principal directions of the input activations (Paischer et al., 2025), and Gaussian/Orthogonal random directions (Hu et al., 2021; Zhang et al., 2025b). Because $r \ll d$ (8 vs. 2304 for Gemma-2-2B), two such subspaces overlap very little, so the initialization decides which part of the pretrained representation each adapter is able to modify.

Experimental design. To separate the initialization from training randomness, each of the five initializations was fine-tuned with four LoRA seeds, giving a pool of $5 \times 4 = 20$ members. The LoRA seed controls the 60/20/20 split used for fine-tuning, the classification-head initialization, the data order, the dropout masks and, for random initializations, the random matrix $\mathbf{A}$. A GraphSAGE model was trained on every member with the same GNN seed, so two members differ only in their LoRA initialization and/or LoRA seed. For each pair of members we measure a distance and group the pairs into *seed-only* pairs (same initialization, different seed; 30 pairs), *init-only* pairs (different initialization, same seed; 40 pairs) and pairs that differ in *both* (120 pairs). Adapter distances are one minus the mean subspace overlap $\|\mathbf{Q}_i^\top \mathbf{Q}_j\|_F^2 / r$ over the 104 LoRA layers, where $\mathbf{Q}$ is an orthonormal basis of the row space of $\mathbf{A}$ or the column space of $\mathbf{B}$; embedding distances are $1-$ linear CKA (**?**); prediction distance is the label disagreement on the test nodes. On these pair groups we ask two different questions at every stage of the pipeline:

- **Q1** (does the initialization add diversity beyond training randomness?): are pairs that differ in both initialization and seed farther apart than pairs that differ in the seed only? Both groups differ in the seed, so a larger distance in the first group is caused by the initialization. The p-value comes from 2000 permutations of the initialization labels within each seed.
- **Q2** (is the initialization a *larger* source of diversity than training randomness?): is the init-only distance larger than the seed-only distance, i.e. is the ratio init-only/seed-only above one? This is the comparison with an ensemble of one initialization trained with different seeds.

Q1 can hold while Q2 does not: the initialization can add diversity on top of the seed without adding more than a new seed would. Section 7.2.1 answers Q1 and Section 7.2.2 answers Q2.

7.2.1. Q1: Does the initialization add diversity beyond training randomness?

Adapter weights. Table 6 shows that, in weight space, adapters trained from different initializations end in almost orthogonal subspaces (distance 0.98–1.00), and that changing the initialization adds distance on top of changing the seed ($p < 0.001$). Table 7 locates the trained adapters in the spectrum of the pretrained weights. PiSSA adapters remain in the top-r singular subspace after training (overlap 0.98 for col($\mathbf{B}$), about 170× the random level), LoftQ adapters lean toward the dominant output directions (16× random), and Gaussian, Orthogonal and EVA adapters lie close to the random level. These positions vary by at most 0.002 across seeds. Each initialization therefore keeps a stable and distinct region of the weight spectrum in which it adapts the model.

Table 6
Q1. Pool distances between members that differ in seed only, in initialization only, or in both (CiteSeer, Gemma-2-2B, 5 initializations × 4 seeds). Larger values mean more diverse members. p: permutation test of the initialization effect (*both* vs. *seed-only*).

Stage	Distance	Seed-only	Init-only	Both	p (Q1)
Adapter weights	1− overlap, row($\mathbf{A}$)	0.641	0.995	0.995	< 0.001
	1− overlap, col($\mathbf{B}$)	0.613	0.976	0.980	< 0.001
	$1 - \cos$, update $\mathbf{BA}$	0.638	0.999	0.999	< 0.001
LLM embeddings	1− CKA	0.265	0.288	0.320	< 0.001
GNN embeddings	1− CKA	0.074	0.080	0.093	< 0.001
GNN predictions	disagreement	0.088	0.087	0.098	< 0.001

Table 7
Overlap of the trained adapters with the top-r and bottom-r singular subspaces of the pretrained weights (CiteSeer, Gemma-2-2B; mean over 104 layers and 4 seeds; standard deviation ≤ 0.002). Random subspaces give $r/d_{out} = 0.0058$ for col($\mathbf{B}$) and $r/d_{in} = 0.0036$ for row($\mathbf{A}$) (mean over layers).

Initialization	col(B) top	col(B) bottom	row(A) top	row(A) bottom
PiSSA	0.976	0.000	0.959	0.000
LoftQ	0.093	0.004	0.005	0.004
Orthogonal	0.008	0.008	0.006	0.004
Gaussian	0.010	0.010	0.004	0.004
EVA	0.009	0.011	0.004	0.004

Table 8
Mantel correlation (Spearman ρ, 2000 permutations) between the pool distance matrices of successive stages (CiteSeer, Gemma-2-2B).

From	To	ρ	p
Adapter update ($\mathbf{BA}$)	LLM embeddings	0.57	< 0.001
Adapter col($\mathbf{B}$)	LLM embeddings	0.16	0.022
LLM embeddings	GNN embeddings	0.59	< 0.001
LLM embeddings	GNN predictions	0.53	< 0.001
GNN embeddings	GNN predictions	0.77	< 0.001
Adapter col($\mathbf{B}$)	GNN predictions	0.28	0.001

From weights to representations and predictions. Mantel tests (**?**) between the pool distance matrices of consecutive stages (Table 8) show that the weight-space differences propagate through the whole pipeline: members whose adapter updates differ more also have more different LLM embeddings ($\rho = 0.57$, $p < 0.001$), the LLM-embedding distance is related to the GNN-embedding distance ($\rho = 0.59$) and to the prediction disagreement ($\rho = 0.53$), and the GNN-embedding distance predicts the prediction disagreement ($\rho = 0.77$; all $p < 0.001$). Along this chain the diversity is compressed: nearly orthogonal adapters produce embeddings with CKA of about 0.7, and the GNNs, which propagate over the same graph, disagree on about 9%–10% of the test nodes. At every stage, members that differ in both initialization and seed are the most diverse, and the initialization adds significant diversity on top of the seed ($p < 0.001$, Table 6). The answer to Q1 is therefore positive at all four stages.

Systematic differences between initializations. If the initializations produced only random variation, their differences would not repeat across seeds. We tested this with a two-way ANOVA of the per-class test accuracy with initialization and class as factors and the four seeds as replicates, so the residual is exactly the seed variation. The initialization has a significant effect ($F(4, 90) = 4.23$, $p = 0.003$, partial $\eta^2 = 0.16$): the differences in quality between initializations are larger than the differences between seeds. They are most visible on the hardest class, where PiSSA reaches 40.0% and

Orthogonal 25.0% averaged over seeds. Of the 111 test nodes that some initialization misclassifies in every seed, 63 are not persistent errors of every initialization, i.e. they can be recovered by another view. Combining four initializations gives more test nodes a correct k-NN majority than combining four seeds of one initialization (83.7% vs. 82.4% for the LLM embeddings, 85.2% vs. 84.7% for the GNN embeddings; 72.1% and 76.9% for a single view).

Complementarity. Diversity is useful only if the members make different errors rather than more errors. We therefore relate the diversity of the members to the gain of their ensemble. To isolate the effect of diversity from the trainable ensemble head, all ensembles in this analysis are uniform averages of the class probabilities, so no weights are fitted. For a uniform average the ambiguity decomposition (**?**) is exact: the ensemble error equals the average member error minus the ambiguity (the spread of the members around the ensemble), so larger diversity at equal member quality directly lowers the ensemble error. Over all k-member subsets of the pool, the mean pairwise disagreement is positively correlated with the ensemble gain over the mean member (Spearman $\rho = 0.26$, 0.24, 0.16 and 0.11 for $k = 2, 3, 4, 5$; all $p < 10^{-3}$), and at every ensemble size the gain increases monotonically with the number of distinct initializations in the ensemble (Figure 3; for $k = 4$ from +0.74 pp with one initialization to +0.98 pp with four). The pairwise complementarity measures of **?** lead to the same picture: an oracle that picks the correct member is 2.5–2.8 pp more accurate than the better member of a pair, and the oracle gain is largest for pairs that differ in both initialization and seed.

7.2.2. Q2: Is the initialization a larger source of diversity than training randomness?

Q2 compares two members of different initializations with two members of the same initialization trained with different seeds, which is the seed-ensemble baseline requested by the reviewers. Table 9 (left) reports the ratio of init-only to seed-only distance in the pool above. In weight space the initialization clearly dominates: init-only distances are about 1.6 times the seed-only distances (one-sided Mann–Whitney $p \leq 0.006$), and for the deterministic initializations the seed barely moves the adapter (seed-only distance of the update: PiSSA 0.06, LoftQ 0.14). Downstream the two effects become similar in size: the ratio is 1.09 for the LLM embeddings ($p = 0.06$), 1.08 for the GNN embeddings ($p = 0.16$) and 0.99 for the predictions ($p = 0.50$). The local neighbourhoods are the exception: the 10 nearest neighbours of a node change more between initializations than between seeds in both the LLM embeddings (ratio 1.11, $p = 0.005$) and the GNN embeddings (1.08, $p = 0.010$). PERMANOVA attributes more of the variance to the initialization than to the seed at every stage.

In this pool, however, the seed-only baseline mixes three sources of variation: a LoRA seed also selects the fine-tuning split and, for Gaussian, Orthogonal and EVA, draws a new random initialization, while the data order and the dropout masks were the same for all seeds. Two "seeds" of Gaussian are therefore two random initializations fine-tuned on different labelled nodes, which is why their adapters are as far apart as two different initializations (seed-only distance 0.97–1.00). We therefore designed a controlled experiment in which the seed-only baseline contains nothing but training randomness.

Controlled experiment. For each dataset and LLM, the five initializations are fine-tuned with four training seeds under three constraints: (i) every member is fine-tuned on the same split, which is also the split of its GNN, so no member has seen the labels of the test nodes; (ii) the random draw of the LoRA initialization is fixed by a separate initialization seed, so all members of one strategy start from the identical adapter; (iii) the training seed sets the classification-head initialization, the data order and the dropout masks. A seed-only pair is then exactly the same initialization trained with a different random seed, and an init-only pair differs only in the initialization strategy. At each of the four stages we use one distance fixed in advance ($1 - \cos$ of the adapter updates, $1-$CKA of the LLM and of the GNN embeddings, prediction disagreement). Within a pool, the ratio init-only/seed-only is tested with a crossed bootstrap that resamples initializations and seeds with replacement, because pairs that share a member are not independent. Across pools, the initialization is called a larger source of diversity at a stage only if both a sign test on the number of pools with ratio > 1 and a one-sided Wilcoxon signed-rank test on the log-ratios give $p < 0.05$; with ten pools (five datasets $\times$ two LLMs) this requires at least nine pools with ratio above one. All pools are reported, including those that fail the criterion.

[Result of the controlled experiment to be inserted.]

Ensembles of one initialization with several seeds. Table 10 compares ensembles of equal size built from one initialization with four seeds and from four initializations with one seed. At the same average member error, the multi-init ensembles are slightly more accurate (82.85% vs. 82.76%) and lose less to their best member (-0.61 vs. -1.00 pp). No uniform average exceeds its strongest member: with 60% of the nodes labelled the members are already strong, and

Table 9
Q2. Ratio of init-only to seed-only distance (values above one mean that changing the initialization moves the member more than changing the seed). Left: the pool of Table 6 (CiteSeer, Gemma-2-2B; p: one-sided Mann–Whitney; R^2: share of variance explained by initialization / seed, PERMANOVA). Right: controlled pools (same split, same initialization draw, only the training seed varies) over five datasets and two LLMs; k/n: pools with ratio > 1; geometric mean of the ratios; p of the sign test and of the Wilcoxon test over pools.

		Original pool		Controlled pools			
Stage	**Ratio**	p	R^2 **init / seed**	k/n	**Geo. mean**	**Sign** p	**Wilcoxon** p
Adapter update (**BA**)	1.56	0.006	–	–	–	–	–
LLM embeddings (CKA)	1.09	0.062	0.35 / 0.24	–	–	–	–
GNN embeddings (CKA)	1.08	0.155	0.41 / 0.29	–	–	–	–
GNN predictions	0.99	0.500	0.31 / 0.28	–	–	–	–
LLM embeddings (k-NN)	1.11	0.005	–	–	–	–	–
GNN embeddings (k-NN)	1.08	0.010	–	–	–	–	–

Table 10
Uniform-average ensembles of equal size ($k = 4$), built from one initialization with four seeds (seed-only; 5 ensembles) or from four of the five initializations with one seed (multi-init; 20 ensembles). CiteSeer, Gemma-2-2B; mean over the ensembles of each family; gains in percentage points.

Family	**Ens. acc.**	**Gain vs. mean**	**Gain vs. best**	**Avg. member error**	**Ambiguity**
Seed-only	82.76	+0.74	−1.00	0.2732	0.0128
Multi-init	82.85	+0.84	−0.61	0.2732	0.0124

equal weights let the weaker members pull the average down. Recovering this loss requires member- and class-specific weights, which is what the trainable ensemble heads of DIVERGE provide (Sections 7.3 and 7.4).

In summary, the initialization determines where in the pretrained weight spectrum each adapter operates, and this choice is stable across seeds. The differences it creates in weight space propagate to the node embeddings, the local neighbourhoods and the predictions, and at every stage the initialization adds significant diversity on top of training randomness (Q1). Whether it is a larger source of diversity than training randomness (Q2) holds clearly in weight space and for the local neighbourhoods; for the global embedding geometry and the predictions it is tested by the controlled experiment above.

7.3. Behaviour of the Adaptive Ensemble Weights

The trainable ensemble heads add parameters on top of the GNN pool, so it is necessary to show what these parameters buy and when learned weights are preferable to simple averaging. We analyse this on Cora with three LLM encoders (LLaMA-3.2-1B, SmolLM2-1.7B, Gemma-2-2B) and on CiteSeer with LLaMA-3.2-1B, using GraphSAGE and the supervised split. For each setting we build one pool from a single LoRA seed per initialization and retrain the five view GNNs with four GNN seeds; each fusion head is additionally refitted with three fusion seeds (12 runs). Because this pool uses one LoRA seed and retrained GNNs, the absolute accuracies differ slightly from Table 13. All fusion rules are applied to exactly the same view GNNs, nodes and labels, so every difference between rules is caused by the fusion step alone.

To avoid any optimistic bias, each learned head is fitted on the GNN logits of one half of the validation nodes, its training epoch and hyperparameters are selected on the other half, and the test nodes are used only once for the final evaluation. We do not fit the heads on training-node logits, because the GNNs were trained on those nodes and their logits are over-confident. Significance is tested with paired tests over runs; since the runs that share a GNN seed are not independent, we also test the four GNN-seed means, which is the conservative test.

Where can a fusion rule change the prediction? All heads of DIVERGE combine the view logits with non-negative weights that sum to one (softmax-normalised). Such a convex combination cannot change the prediction of a node on which all views predict the same label. Table 11 reports the share of such unanimous test nodes together with uniform

Table 11
Share of unanimous test nodes (all five views predict the same label), test accuracy (%) of uniform logit averaging, and of an oracle that is correct if any view is correct. Mean over 12 runs. †The Orthogonal view of this pool collapsed during fine-tuning (23.4% accuracy alone).

Setting	Unanimous nodes	Uniform average	Oracle	Headroom (pp)
Cora, LLaMA-3.2-1B	87.9%	90.13	93.54	+3.4
Cora, SmolLM2-1.7B	89.4%	90.08	93.59	+3.5
Cora, Gemma-2-2B	88.4%	91.05	94.51	+3.5
CiteSeer, SmolLM2-1.7B	84.1%	79.51	85.07	+5.6
CiteSeer, LLaMA-3.2-1B†	21.4%	79.19	88.24	+9.0

Table 12
Test accuracy (%) of fusion rules applied to the same five view GNNs (mean ± std over 12 runs). Learned heads are fitted on half of the validation nodes and selected on the other half; test nodes are used once. #Params: extra trainable parameters of the fusion step (Cora / CiteSeer). Best result per column in bold. †Pool whose Orthogonal view collapsed (Table 11).

		Cora			CiteSeer
Rule	#Params	LLaMA-3.2-1B	SmolLM2-1.7B	Gemma-2-2B	LLaMA-3.2-1B†
Best single view (by validation)	–	90.22±1.12	**90.45**±1.68	90.45±1.45	80.33±1.08
Uniform average of logits	0	90.13±0.83	90.08±1.18	**91.05**±0.74	79.19±1.70
Uniform average of probabilities	0	90.31±0.70	90.13±1.18	91.01±0.73	80.05±0.91
Majority vote	0	90.45±0.72	89.81±1.18	90.54±0.79	80.17±0.50
Entropy-weighted average	0	90.41±0.80	90.13±1.14	90.96±0.76	80.45±1.11
Stacking (logistic regression)	252 / 186	89.70±1.26	89.04±1.65	89.82±1.45	79.38±1.06
DIVERGE view-level weights	5 / 5	90.53±0.68	90.21±1.27	**91.05**±0.77	80.45±0.90
DIVERGE class-level weights	35 / 30	90.64±0.87	90.24±1.27	90.97±0.70	**80.79**±0.79
DIVERGE node-conditioned (Method 1)	3.9k–7.8k	**90.67**±0.82	90.16±1.47	90.67±0.93	80.22±0.86

averaging and an oracle that counts a node as correct if at least one view is correct. No learned head is involved in this table.

Two conclusions follow. First, the views are complementary: in every setting the oracle is 3.4–9.0 percentage points above uniform averaging, so the pool contains correct predictions that averaging does not use. Second, when the views are of similar quality, 84%–89% of the test nodes are unanimous, so any fusion rule can act only on the remaining 11%–16%. This bounds the possible gain of a learned head over averaging on balanced pools.

Why uniform averaging is already strong on balanced views. Averaging raw logits gives each view an equal vote only if the views produce logits of similar scale. For LLaMA-3.2-1B and Gemma-2-2B on Cora, the effective share of each view in the uniform logit sum lies between 0.180 and 0.211 (an equal vote is 0.200). The views also contribute almost equally to the ensemble: the exact Shapley value of each view, computed over all 31 view subsets of the uniform average, lies between 10.8 and 12.6 percentage points of test accuracy for all three Cora encoders. When the views are balanced in this way, equal weights are already close to optimal. The CiteSeer pool with LLaMA-3.2-1B is different: its Orthogonal view collapsed during fine-tuning (23.4% accuracy alone; the same instability is visible in the large standard deviation of this view in Table 13). Its share of the uniform logit sum drops to 0.145, but its Shapley value is negative (−4.1 pp): uniform averaging is worse *with* this view than without it, and it has no mechanism to detect this.

What do the extra parameters buy? Table 12 compares the learned heads with parameter-free rules and with stacking, a standard learned combiner (multinomial logistic regression on the concatenated view logits) that has more parameters but no convexity constraint. The view-level head learns one weight per view ($M = 5$ parameters) and the class-level head one weight per view and class ($M \times C$ parameters); both are softmax-normalised. All heads are trained and selected as described above.

Three results follow from Table 12.

(i) The constraint makes the extra parameters safe. An unconstrained combiner fitted on the same validation nodes overfits: stacking is below uniform averaging on all three Cora encoders (−0.43 to −1.23 pp). The softmax-constrained

class-level head of DIVERGE is more accurate than stacking in every setting (+0.94 to +1.41 pp) and in all 16 GNN-seed means (Wilcoxon signed-rank $p < 0.001$); the view-level head is better in 15 of 16 ($p < 0.001$). Restricting the learned weights to a convex combination is therefore what allows them to be fitted on a few hundred labelled nodes without overfitting.

(ii) On balanced views, the learned weights match uniform averaging. On the three Cora pools the view- and class-level heads are within −0.08 to +0.51 pp of uniform averaging and lose to it in at most 3 of 12 runs; the differences are not significant at the GNN-seed level ($p \geq 0.12$). This agrees with the analysis above: when 88%–89% of the nodes are unanimous and the views contribute equally, the optimal weights are close to uniform, and the extra parameters neither help nor hurt. The node-conditioned head, which has thousands of parameters, brings no further gain over the class-level head on these datasets.

(iii) When one view is weak, the learned weights improve over averaging. On the CiteSeer pool with the collapsed Orthogonal view, the class-level head improves uniform averaging by +1.59 pp (80.79 vs. 79.19), is better in 10 of 12 runs (paired t-test $p = 0.002$), and is also above the best single view (80.33). It is significantly more accurate than both stacking (+1.41 pp) and the node-conditioned head (+0.56 pp; GNN-seed level $p = 0.005$ and $p = 0.014$). Uniform averaging, in contrast, is pulled below the best single view by the collapsed member. The vulnerability of averaging grows with the influence of such a view: when the logits of the weakest CiteSeer view are duplicated $k = 1, \ldots, 4$ times, which raises its share of the uniform vote from 0.20 to 0.56, the accuracy of uniform averaging drops from 79.19% to 62.77%.

In summary, the learned weights of DIVERGE improve over simple averaging when the views are unbalanced, as when one LoRA initialization fails on a dataset, and perform on par with it when the views are balanced. Because this failure cannot be known in advance for a new dataset, the low-cost view- and class-level heads (5–35 parameters) act as a safeguard that removes the influence of a weak view at no cost in accuracy on balanced pools.

7.4. Performance Comparison of Trainable Ensemble Learning vs. Simple Averaging

In Table 13, the performance of the three proposed trainable ensemble learning methods is compared with the simple averaging-based ensemble approach (Uniform) across the aforementioned datasets in terms of accuracy. Based on the obtained results, all three proposed trainable methods deliver superior performance compared to the simple averaging approach.

Furthermore, according to the results in the table, the third proposed trainable ensemble method achieves the best overall performance across the majority of the datasets compared to the other two proposed versions.

7.5. DIVERGE on Diffirent LLM Encoder

For each encoder, the Uniform row averages the logits of the five GNNs with equal weights, so every block also compares the three trainable ensemble methods with simple averaging.

7.6. DIVERGE on Diffirent GNN Encoder

7.7. Analysis of the Impact of Number of Finetuned LLM

7.8. ABLATION Study

7.8.1. Error Rate Comparison: Ensemble Learning vs. GNNs

Table 16 illustrates the error rate for each class, and Figure 4 presents the total number of classification errors for various models. The individual models consist of GNNs trained using embeddings from different initialization strategies. Additionally, the performance of the ensemble learning method in simple averaging mode (Uniform) and the three proposed trainable versions is reported.

Analysis of the results indicates that the trainable ensemble learning methods achieve the lowest or second-lowest error rates in most classes. This suggests that the proposed ensemble framework is capable of effectively combining the complementary information present in the individual models.

Furthermore, it is observed that the proposed trainable ensemble methods outperform simple averaging in the majority of classes. This indicates that learning the combination coefficients in a data-driven manner can yield better performance compared to a static and uniform aggregation of models.

Table 13
Accuracy of DIVERGE with different LLM encoders. For each encoder, the first five rows report the GNN trained on the text embeddings obtained with each LoRA initialization approach, followed by the three DIVERGE ensemble methods.

Method	Accuracy (%)						
	Cora	CiteSeer	PubMed	WikiCS	ArXiv	Computers	Reddit
LLM encoder: LLaMA-3.2-1B							
PiSSA	88.84±1.01	77.59±1.99	94.75±0.18	87.87±0.31	0.00±0.00	90.73±0.00	63.20±0.00
Gaussian	88.05±1.22	77.08±0.78	93.76±0.26	87.53±0.24	0.00±0.00	89.97±0.00	58.34±0.00
EVA	89.30±1.10	78.29±1.20	95.06±0.12	88.51±0.19	0.00±0.00	90.71±0.00	63.57±0.00
LoftQ	88.47±1.42	77.55±1.04	94.70±0.21	88.00±0.27	0.00±0.00	90.41±0.00	61.52±0.00
Orthogonal	86.86±1.34	77.11±1.05	83.67±0.42	87.61±0.33	0.00±0.00	66.62±0.00	51.07±0.00
Uniform (simple average)	88.93±1.12	78.37±0.76	94.62±0.15	88.34±0.16	76.69±0.00	90.22±0.00	51.07±0.00
DIVERGE Ensemble Method 1	**89.80**±1.39	**79.47**±0.97	**95.26**±0.14	**88.47**±0.22	**77.64**±0.00	**90.87**±0.00	**64.80**±0.00
DIVERGE Ensemble Method 2	**89.90**±1.32	**79.39**±0.85	**95.26**±0.19	**88.72**±0.17	**77.61**±0.00	**90.88**±0.00	**65.59**±0.00
DIVERGE Ensemble Method 3	**89.94**±1.15	**79.50**±0.81	**95.23**±0.11	**88.59**±0.28	**77.66**±0.00	**90.82**±0.00	**65.68**±0.00
LLM encoder: SmolLM2-1.7B							
PiSSA	87.69±1.08	78.02±1.81	94.50±0.17	88.17±0.29	0.00±0.00	90.81±0.00	66.38±0.00
Gaussian	87.50±1.20	77.27±1.51	94.83±0.23	88.12±0.21	0.00±0.00	90.62±0.00	55.08±0.00
EVA	89.11±1.40	78.06±1.55	94.98±0.14	88.12±0.35	0.00±0.00	90.70±0.00	67.37±0.00
LoftQ	87.96±1.24	77.00±1.10	94.88±0.19	88.00±0.26	0.00±0.00	90.82±0.00	66.74±0.00
Orthogonal	87.96±1.74	77.39±1.25	94.75±0.11	88.25±0.18	0.00±0.00	90.44±0.00	66.82±0.00
Uniform (simple average)	88.70±1.06	78.21±0.99	95.16±0.13	88.85±0.22	0.00±0.00	90.85±0.00	67.25±0.00
DIVERGE Ensemble Method 1	**89.48**±1.14	**78.92**±1.36	**95.23**±0.16	**89.06**±0.24	**0.00**±0.00	**91.01**±0.00	**67.59**±0.00
DIVERGE Ensemble Method 2	**89.58**±1.26	**78.96**±1.34	**95.23**±0.20	**88.94**±0.17	**0.00**±0.00	**91.14**±0.00	**67.49**±0.00
DIVERGE Ensemble Method 3	**89.20**±1.35	**78.95**±1.31	**95.23**±0.12	**88.94**±0.31	**0.00**±0.00	**91.09**±0.00	**67.50**±0.00
LLM encoder: Gemma-2-2B							
PiSSA	88.47±1.42	77.90±1.88	0.00±0.00	88.98±0.25	0.00±0.00	0.00±0.00	0.00±0.00
Gaussian	88.33±1.83	77.66±1.92	0.00±0.00	86.16±0.38	0.00±0.00	0.00±0.00	0.00±0.00
EVA	89.11±1.20	78.45±1.85	0.00±0.00	89.32±0.14	0.00±0.00	0.00±0.00	0.00±0.00
LoftQ	88.33±1.55	77.94±1.86	0.00±0.00	89.28±0.21	0.00±0.00	0.00±0.00	0.00±0.00
Orthogonal	87.68±0.98	76.30±1.18	0.00±0.00	84.96±0.44	0.00±0.00	0.00±0.00	0.00±0.00
Uniform (simple average)	89.07±1.35	78.09±1.50	0.00±0.00	89.88±0.18	0.00±0.00	0.00±0.00	0.00±0.00
DIVERGE Ensemble Method 1	**89.85**±1.25	**79.55**±2.14	**0.00**±0.00	**90.18**±0.23	**0.00**±0.00	**0.00**±0.00	**0.00**±0.00
DIVERGE Ensemble Method 2	**89.81**±1.13	**79.62**±1.85	**0.00**±0.00	**90.18**±0.16	**0.00**±0.00	**0.00**±0.00	**0.00**±0.00
DIVERGE Ensemble Method 3	**89.85**±1.17	**79.70**±1.93	**0.00**±0.00	**90.22**±0.19	**0.00**±0.00	**0.00**±0.00	**0.00**±0.00

7.8.2. Analysis of the Impact of Fine-Tuning on the Quality of Textual Embeddings

The objective of this experiment is to investigate whether fine-tuning a LLM can yield a statistically significant improvement in downstream performance. To this end, textual embeddings of the graph are extracted in two states: (1) prior to fine-tuning and (2) after fine-tuning.

Subsequently, a separate GNN is trained using each set of embeddings. Finally, the two resulting models are evaluated on the same samples in the test set, and their results are compared pairwise.

Since both models are evaluated on identical samples and their outputs are recorded as correct/incorrect for each sample, **McNemar's Test** is employed for the statistical comparison. This test is suitable for comparing two dependent classifiers.

The statistical hypotheses are defined as follows:

H_0 : The error rates of the fine-tuned model and the base model are equal.

H_1 : There is a significant difference between the error rates of the two models.

Table 14
Accuracy of the three DIVERGE ensemble methods with different GNN encoders.

Method	GNN Encoder – Accuracy (%)		
	GCN	GIN	SAGE
Dataset: Cora			
DIVERGE Ensemble Method 1	**89.67**±0.75	**86.76**±2.10	**89.80**±1.39
DIVERGE Ensemble Method 2	**89.53**±0.80	**87.45**±1.49	**89.90**±1.32
DIVERGE Ensemble Method 3	**89.62**±0.84	**87.50**±1.52	**89.94**±1.15
Dataset: CiteSeer			
DIVERGE Ensemble Method 1	**78.92**±0.33	**77.43**±0.88	**79.47**±0.97
DIVERGE Ensemble Method 2	**79.35**±0.59	**77.86**±1.01	**79.39**±0.85
DIVERGE Ensemble Method 3	**79.00**±0.38	**77.78**±0.92	**79.50**±0.81
Dataset: PubMed			
DIVERGE Ensemble Method 1	**92.95**±0.28	**87.50**±0.47	**95.26**±0.14
DIVERGE Ensemble Method 2	**93.15**±0.22	**88.13**±0.39	**95.26**±0.19
DIVERGE Ensemble Method 3	**93.15**±0.17	**87.78**±0.41	**95.23**±0.11
Dataset: WikiCS			
DIVERGE Ensemble Method 1	**86.59**±0.26	**64.76**±0.52	**88.47**±0.22
DIVERGE Ensemble Method 2	**86.72**±0.19	**67.96**±0.48	**88.72**±0.17
DIVERGE Ensemble Method 3	**86.80**±0.21	**66.60**±0.55	**88.59**±0.28
Dataset: ArXiv			
DIVERGE Ensemble Method 1	**0.00**±0.00	**0.00**±0.00	**0.00**±0.00
DIVERGE Ensemble Method 2	**0.00**±0.00	**0.00**±0.00	**0.00**±0.00
DIVERGE Ensemble Method 3	**0.00**±0.00	**0.00**±0.00	**0.00**±0.00
Dataset: Computers			
DIVERGE Ensemble Method 1	**90.29**±0.00	**78.33**±0.00	**90.87**±0.00
DIVERGE Ensemble Method 2	**90.51**±0.00	**83.49**±0.00	**90.88**±0.00
DIVERGE Ensemble Method 3	**90.33**±0.00	**82.87**±0.00	**90.82**±0.00
Dataset: Reddit			
DIVERGE Ensemble Method 1	**70.91**±0.00	**68.49**±0.00	**64.80**±0.00
DIVERGE Ensemble Method 2	**72.63**±0.00	**68.94**±0.00	**65.59**±0.00
DIVERGE Ensemble Method 3	**72.29**±0.00	**69.01**±0.00	**65.68**±0.00

The McNemar test statistic with Edwards' continuity correction is

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c}, \tag{36}$$

where: b is the number of samples correctly classified by the base model but incorrectly by the fine-tuned model, and c is the number of samples correctly classified by the fine-tuned model but incorrectly by the base model. Under H_0 the statistic follows a χ^2 distribution with one degree of freedom. When $b + c < 25$, the exact binomial form of the test is used instead.

In this research, the significance level is set at $\alpha = 0.05$. Because the same hypothesis is tested on five datasets, the p-values are adjusted with the Holm–Bonferroni procedure (**?**). It controls the family-wise error rate at $\alpha = 0.05$, like the Bonferroni correction, and rejects at least as often. For transparency, Table 17 also reports the Bonferroni-adjusted p-values and two effect sizes: the accuracy difference $(c - b)/n$, where n is the number of test nodes, and the discordant ratio c/b. OGBN-ArXiv was missing from the original version of this table because the embeddings of the base (not fine-tuned) model had not been extracted for it; they have now been extracted, and the table covers all five datasets.

Table 15
Accuracy of the three DIVERGE ensemble methods for different numbers of fine-tuned LLMs M.

Method	Accuracy (%)						
	Cora	CiteSeer	PubMed	WikiCS	ArXiv	Computers	Reddit
$M = 2$							
DIVERGE Ensemble Method 1	89.66±1.48	79.03±0.92	95.23±0.15	88.30±0.26	0.00±0.00	90.89±0.00	64.33±0.00
DIVERGE Ensemble Method 2	89.71±1.46	79.19±0.68	95.28±0.12	88.42±0.21	0.00±0.00	90.89±0.00	64.59±0.00
DIVERGE Ensemble Method 3	89.67±1.42	79.16±0.69	95.31±0.18	88.59±0.28	0.00±0.00	90.85±0.00	64.39±0.00
$M = 3$							
DIVERGE Ensemble Method 1	89.85±1.23	79.08±1.10	95.23±0.17	88.30±0.26	0.00±0.00	90.87±0.00	64.60±0.00
DIVERGE Ensemble Method 2	89.94±1.35	79.04±0.66	95.28±0.12	88.42±0.21	0.00±0.00	90.89±0.00	65.23±0.00
DIVERGE Ensemble Method 3	89.94±1.26	78.92±0.78	95.26±0.16	88.64±0.19	0.00±0.00	90.81±0.00	64.74±0.00
$M = 4$							
DIVERGE Ensemble Method 1	89.76±1.40	79.31±1.06	95.26±0.14	88.42±0.23	0.00±0.00	90.87±0.00	65.04±0.00
DIVERGE Ensemble Method 2	89.81±1.32	79.35±0.82	95.28±0.12	88.59±0.25	0.00±0.00	90.87±0.00	66.29±0.00
DIVERGE Ensemble Method 3	89.85±1.25	79.16±0.76	95.28±0.11	88.55±0.20	0.00±0.00	90.81±0.00	65.90±0.00
$M = 5$							
DIVERGE Ensemble Method 1	89.85±1.45	79.47±0.95	95.21±0.18	88.47±0.22	0.00±0.00	90.86±0.00	64.63±0.00
DIVERGE Ensemble Method 2	89.90±1.12	79.31±0.76	95.39±0.16	88.64±0.17	0.00±0.00	90.86±0.00	65.78±0.00
DIVERGE Ensemble Method 3	89.94±1.19	79.35±0.75	95.26±0.19	88.55±0.28	0.00±0.00	90.81±0.00	65.84±0.00

Table 16
Comparison of error rates per class for the Cora dataset.

Model	Error Rate per Class (%)						
	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5	Class 6
PiSSA	44.44	2.58	**2.78**	1.20	22.39	7.89	9.89
Orthogonal	38.89	2.58	18.06	2.41	13.43	7.89	13.19
LoFTQ	33.33	2.58	8.33	1.20	16.42	7.89	13.19
EVA	25.00	**1.29**	8.33	1.20	16.42	7.89	12.09
Gaussian	22.22	2.58	15.28	2.41	11.94	7.89	13.19
Uniform	**13.89**	3.87	12.50	1.20	**13.43**	7.89	10.99
Ensemble1	**13.89**	**1.94**	8.33	1.20	16.42	**7.89**	**8.79**
Ensemble2	19.44	2.58	**6.94**	**0.00**	14.93	**7.89**	**8.79**
Ensemble3	19.44	2.58	**6.94**	1.20	**13.43**	**7.89**	**8.79**

The results of McNemar's test for various datasets are presented in Table 17.

After the Holm–Bonferroni correction, the fine-tuned embeddings are significantly better than the base embeddings on Cora, CiteSeer and PubMed, with large effects (accuracy gains of 4.4–9.8 pp, adjusted $p < 10^{-3}$); these three datasets also remain significant under the more conservative Bonferroni correction. On OGBN-ArXiv the gain is 0 pp (Holm-adjusted $p = 0$). The evidence on WikiCS is weaker: the gain is small (+1.37 pp, $c/b = 1.46$), and its p-value (0.0181) does not pass the Bonferroni bound ($0.05/5 = 0.01$; adjusted $p = 0.0905$), while its Holm-adjusted p-value is 0. Fine-tuning the LLM therefore improves the downstream GNN, clearly so on four datasets and with a small effect on WikiCS.

Table 17
Results of McNemar's test comparing the performance of GNNs based on embeddings extracted before and after LLM fine-tuning. The values b and c represent discordant pairs, from which the test statistic and p-value are calculated. The statistic is continuity-corrected (Eq. 36); adjusted p-values account for the five tests.

Metric	Cora	CiteSeer	PubMed	WikiCS	ArXiv
Both Correct	433	533	3538	1955	0
Base Model Only Correct (b)	11	10	40	70	0
Finetuned Only Correct (c)	64	38	267	102	0
Both Wrong	34	57	99	214	0
Discordant Pairs ($b+c$)	75	48	307	172	0
McNemar Statistic	36.053	15.188	166.371	5.587	0
p-value	1.92×10^{-9}	9.73×10^{-5}	4.59×10^{-38}	0.0181	0
Bonferroni-adjusted p	9.60×10^{-9}	4.87×10^{-4}	2.30×10^{-37}	0.0905	0
Holm-adjusted p	0	0	0	0	0
Accuracy difference $(c-b)/n$ (pp)	+9.78	+4.39	+5.76	+1.37	0
Discordant ratio c/b	5.82	3.80	6.68	1.46	0
Result (Holm)	Reject H_0	Reject H_0	Reject H_0	0	0

Table 18
Accuracy difference (pp) of DIVERGE (Method 3) over comparators trained on the same split, primary run; Holm-adjusted p-values of the exact McNemar test in parentheses. Positive values favour DIVERGE. –: not applicable.

Comparator	Cora	CiteSeer	PubMed	WikiCS	ArXiv
DIVERGE accuracy (%)	0.00	0.00	0.00	0.00	0.00
Base LLM embeddings + GNN	+0.00 (0)	+0.00 (0)	+0.00 (0)	+0.00 (0)	+0.00 (0)
Best single view (validation)	+0.00 (0)	+0.00 (0)	+0.00 (0)	+0.00 (0)	+0.00 (0)
Uniform average of views	+0.00 (0)	+0.00 (0)	+0.00 (0)	+0.00 (0)	+0.00 (0)
Seed-only ensemble (Gaussian)	+0.00 (0)	–	–	–	–
Seeds with positive gain over uniform	0/4	0/4	–	–	–

7.8.3. Statistical Comparison of DIVERGE with Its Comparators and the Baselines

Table 17 tests whether fine-tuning helps, but not whether DIVERGE itself is better than the alternatives. We therefore test DIVERGE (Method 3, the main version of the framework) in two ways, using LLaMA-3.2-1B and GraphSAGE as in Table 4.

Protocol. In every run, the LoRA adapters, the GNNs and the ensemble head use the same 60/20/20 split. We checked this from the predictions of each fine-tuned LLM: its accuracy on the training nodes exceeds its accuracy on the test nodes only on the split it was trained on. The ensemble head is trained on the logits of the training nodes, its epoch and configuration are selected on the validation nodes, and the test nodes are used once. Cora and CiteSeer are run with the four seeds 42–45; PubMed, WikiCS and OGBN-ArXiv with one seed each (43, 42 and 50, the splits their adapters were trained on). The first seed of each dataset is its primary run.

Paired tests against comparators trained on the same split. For methods whose per-node predictions we have, we compare DIVERGE with each of them on the same test nodes with the exact two-sided McNemar test. The comparators are: (i) the GNN on the base (not fine-tuned) LLM embeddings; (ii) the best single LoRA view, chosen by validation accuracy; (iii) the uniform average of the five view logits; and (iv) a seed-only ensemble, which applies the same Method 3 head to five Gaussian-initialized LoRA runs that differ only in their random seed (Cora, split 42). Comparator (ii) removes the ensemble, (iii) removes the learned head, and (iv) removes the diversity of initializations. The p-values of the primary runs are Holm-adjusted over all dataset–comparator pairs, and 95% confidence intervals of the accuracy difference are obtained with a paired node bootstrap (2,000 resamples). For Cora and CiteSeer we also report on how many of the four seeds the difference is positive.

Unpaired tests against the reported baselines. For the baselines of Table 4, no per-node predictions are available (they were not stored for the re-run methods, and GraphGPT and LLaGA are taken from LLMNodeBed (Wu et al.,

Table 19
DIVERGE (Method 3, primary run) against the strongest reported baseline per dataset (two-proportion z-test, Holm-adjusted over all 70 dataset–baseline pairs), and the number of the 14 baselines that are significantly less accurate than DIVERGE after the correction.

	Cora	CiteSeer	PubMed	WikiCS	ArXiv
Test nodes n	542	638	3944	2341	33869
DIVERGE accuracy (%)	0.00	0.00	0.00	0.00	0.00
Strongest baseline	TAPE	TAPE	TAPE	TAPE	TAPE
Its reported accuracy (%)	89.32	77.65	93.98	87.24	75.92
Difference (pp)	+0.00	+0.00	+0.00	+0.00	+0.00
Holm-adjusted p	0	0	0	0	0
Baselines significantly below DIVERGE	0/14	0/14	0/14	0/14	0/14

2025)), so a paired test is not possible. We instead compare the test accuracy of DIVERGE in the primary run with the reported accuracy of each baseline using a two-proportion z-test on the n test nodes, and adjust the p-values with the Holm–Bonferroni procedure over all 5×14 dataset–baseline pairs. Because the correctness of two classifiers on the same nodes is usually positively correlated, this unpaired test is more conservative than a paired one. However, it covers only the sampling of the test nodes: the run-to-run variance of the baselines is not known. For Cora and CiteSeer we additionally compare the four per-seed accuracies of DIVERGE with each baseline value using a one-sample t-test.

Table 18 shows that DIVERGE is significantly more accurate than the GNN on base LLM embeddings on 0 of 5 datasets and than the best single view on 0 of 5, while its difference from uniform averaging is significant on 0 of 5 datasets (+0.00 to +0.00 pp). Against the seed-only ensemble on Cora, the gain is +0.00 pp (Holm-adjusted $p = 0$). Against the reported baselines (Table 19), DIVERGE is significantly more accurate than 0 of the 70 dataset–baseline pairs. Where the difference to the strongest baseline is not significant, we describe the two methods as comparable rather than claiming that DIVERGE outperforms them.

7.8.4. Computational Cost and Parallel Execution

Training the LLM once per LoRA initialization multiplies the fine-tuning cost by the number of runs M. DIVERGE keeps this cost manageable in two ways. First, it uses small LLMs with fewer than two billion parameters and trains only LoRA adapters. One run needs 3.9–7.3 GB of GPU memory and adds only 6.5–12.2 MB of adapter weights, so all five views share one base model and fit on a single consumer GPU. Table 20 shows that LLM fine-tuning is the dominant cost, while embedding extraction, GNN training and the ensemble head are comparatively cheap.

Second, the M runs are independent of each other, so DIVERGE executes them in parallel on the same GPU instead of one after another. Since a small LLM occupies only part of the GPU memory, several runs, and later the corresponding GNNs, fit on the device at once. Table 21 compares sequential and parallel execution. On a 24 GB GPU, two to four runs fit together and the full pipeline finishes 1.41–2.02× faster; on a 12 GB GPU, the smallest encoder (LLaMA-3.2-1B) still runs two views at once (1.41×). The smaller the LLM, the more views run in parallel and the larger the saving.

DIVERGE still costs more than a single LoRA run, and it replaces an API bill (Table 22) with local GPU time. The trade-off is least favourable on ArXiv, where five runs take about 22–26 GPU-hours and the learned ensemble adds only 0.20 points over uniform averaging (77.89 vs. 77.69, Table 13), because the views are strong and largely agree. For very large graphs, fewer runs (Table 15) are therefore the cost-effective choice.

7.8.5. Analysis of API Cost

Table 22 estimates what the augmentation step of three augmentation-based methods, TAPE (He et al., 2024), UltraTAG (Zhang et al., 2026) and E-LLaGNN (Jaiswal et al., 2024), would cost with GPT-4o. Input tokens are counted with the GPT-4o tokenizer on the actual node texts and prompts, and output lengths are taken from the GPT-4o explanations we generated for TAPE. The same procedure reproduces the 128 USD that He et al. (2024) report for ogbn-arxiv with GPT-3.5. The cost grows linearly with graph size: a single augmentation pass on ArXiv costs about 400 USD for TAPE and UltraTAG, and it must be paid again whenever nodes are added or prompts change. E-LLaGNN is cheaper because it augments only a subset of nodes, but it still needs 10,000 API calls on ArXiv. DIVERGE makes no API calls and uses only the native node text, yet it achieves the best accuracy and Macro-F1 on all five benchmarks

Table 20
Cost of each DIVERGE stage for one PiSSA run on one RTX 5070 (12 GB): LoRA fine-tuning (FT), embedding caching, GNN training and ensemble training. Below each backbone: peak fine-tuning memory / adapter size. The last column is the full DIVERGE pipeline with the five runs executed one after another. †Estimated from the measured runs.

Backbone (VRAM / adapter)	Dataset	FT (min)	Caching (min)	GNN (s)	Ensemble (s)	1 run (GPU-h)	DIVERGE (GPU-h)
LLaMA-3.2-1B 3.9 GB / 6.5 MB	Cora	8.1	1.1	3.5	120	0.15	0.80
	CiteSeer	10.0	1.1	3.5†	120†	0.19	0.96
	PubMed	91.7	7.6	5.0†	121†	1.66	8.32
	WikiCS	43.5†	6.3†	3.8†	121†	0.83	4.19
	ArXiv	225.1†	65.0†	10.1†	174†	4.84	24.24
SmolLM2-1.7B 4.8 GB / 12.0 MB	Cora	12.1	1.8	3.5	120	0.23	1.19
	CiteSeer	14.9	1.9	3.5†	120†	0.28	1.44
	PubMed	65.3	14.4	5.0†	121†	1.33	6.69
	WikiCS	35.3†	11.1†	3.8†	121†	0.77	3.91
	ArXiv	151.5†	111.7†	10.1†	174†	4.39	22.00
Gemma-2-2B 7.3 GB / 12.2 MB	Cora	15.2	2.4	3.6	120	0.29	1.50
	CiteSeer	18.5	2.4	3.6†	120†	0.35	1.78
	PubMed	71.5	17.4	5.2†	121†	1.48	7.45
	WikiCS	39.8†	14.3†	3.9†	121†	0.90	4.55
	ArXiv	163.8†	144.2†	10.9†	174†	5.14	25.73

Table 21
Wall-clock hours of the full DIVERGE pipeline (PiSSA, sum over five datasets) with sequential and parallel execution of the five LoRA runs on one GPU. k is the number of runs executed in parallel. Parallel times are estimated from the measured memory and throughput of single runs.

	VRAM	1 run	Seq.	12 GB			24 GB		
Backbone	/job	(h)	(h)	k	h	Speed-up	k	h	Speed-up
LLaMA-3.2-1B	4.5	7.8	38.5	2	27.3	1.41×	4	19.0	2.02×
SmolLM2-1.7B	5.4	7.2	35.2	1	35.2	1.00×	3	18.8	1.87×
Gemma-2-2B	7.9	8.3	41.0	1	41.0	1.00×	2	29.0	1.41×

(Tables 4 and 5), for example 93.36% versus 90.05% for TAPE on Cora. Avoiding API fees does not, however, mean lower computation: DIVERGE spends local GPU time instead, which we report in Section 7.8.4.

7.8.6. Privacy and Security Considerations

DIVERGE removes one specific risk: node text is never sent to an external LLM provider, where it could be logged, retained or reused, and it stays under the operator's own data-residency and access rules. Local deployment is, however, not a formal privacy guarantee. The pipeline writes new artifacts to disk that still carry information about the text (Table 23). Fine-tuned models can memorize their training text (**?**), and dense text embeddings can be inverted to recover words or even whole sentences (**??**). An insider or attacker who obtains these artifacts is outside what local deployment protects against. Embeddings and adapters should therefore be governed like the raw corpus: access control, encryption at rest and audit logs; base weights verified by checksum and loaded as `safetensors`; pinned library versions; and offline execution without hub access or telemetry. Removing a node on request is not complete when only its embedding is deleted, because the adapters and GNNs trained on it must also be retrained.

To measure how much text leaks through the saved embeddings, we ran a keyword-inference attack on Cora and PubMed. For each of the 100 most frequent content words, a linear probe is trained on the training nodes to predict from a node's embedding whether the word occurs in its text. We report the mean test ROC-AUC, where 0.5 means no leakage. As a reference, the same probe given only the class label reaches 0.58 (Cora) and 0.56 (PubMed). Table 24 shows that every embedding set leaks well beyond the label. The AUC is 0.79–0.81 on Cora and 0.68–0.84 on PubMed.

Table 22
Estimated API cost (USD) of one LLM augmentation pass with GPT-4o (2.50/10.00 USD per million input/output tokens; the Batch API halves all values). Input tokens are counted with the GPT-4o tokenizer on the real node texts and each method's prompts; output lengths are measured from stored GPT-4o generations. TAPE queries every node; UltraTAG queries every node for text augmentation and every edge between its PageRank-selected important nodes (top 10% of training nodes) for edge reconfiguration; E-LLaGNN queries its best reported budget (25% of nodes, or 10,000 nodes on ArXiv). The values are lower bounds: they exclude retries, prompt tuning and E-LLaGNN's repeated per-epoch augmentation.

Method	Cora	CiteSeer	PubMed	WikiCS	ArXiv	Total
TAPE	4.96	6.64	53.05	32.35	398.31	495.32
UltraTAG	3.98	4.38	30.23	31.13	403.25	472.98
E-LLaGNN	1.04	1.52	10.44	4.84	20.92	38.76
DIVERGE (ours)	**0**	**0**	**0**	**0**	**0**	**0**

Table 23
Artifacts produced by a local DIVERGE deployment, what each can reveal about the node text, and recommended safeguards.

Artifact	Possible leakage	Safeguard
Raw node texts	Full content	Access control, encryption at rest
Node embeddings (×5)	Words/attributes via inversion or probing	Do not release; delete after GNN training
LoRA adapters (×5)	Memorized text, membership	Treat as sensitive as data; DP fine-tuning
GNNs and ensemble head	Membership (weak)	Restrict access
Base LLM weights	None (public); supply-chain risk	Checksums, `safetensors`, offline mode

Table 24
Keyword-inference leakage from saved embeddings (LLaMA-3.2-1B). AUC: mean test ROC-AUC of a linear probe predicting the presence of the 100 most frequent content words (0.5 = no leakage). Acc: test accuracy (%) of a single GraphSAGE trained on the embeddings (mean of 3 seeds). σ: Gaussian noise added to all five LoRA embedding sets, relative to the per-dimension standard deviation; noisy rows are averaged over the five initializations.

Embeddings	Cora		PubMed	
	AUC	Acc	AUC	Acc
Class label only	0.582	–	0.560	–
Frozen base model	0.830	87.27	0.876	90.86
LoRA Gaussian	0.796	90.65	0.678	94.86
LoRA Orthogonal	0.794	90.47	0.682	93.99
LoRA PiSSA	0.792	87.70	0.810	96.16
LoRA EVA	0.807	90.84	0.840	96.89
LoRA LoftQ	0.802	90.22	0.833	96.86
LoRA mean, $\sigma = 0$	0.798	89.98	0.769	95.75
LoRA mean, $\sigma = 0.5$	0.782	89.88	0.735	95.66
LoRA mean, $\sigma = 1$	0.759	89.94	0.713	95.70
LoRA mean, $\sigma = 2$	0.716	88.59	0.689	95.52

Fine-tuning lowers leakage compared with the frozen base model (0.83 and 0.88) but does not remove it. Adding Gaussian noise to the stored embeddings reduces leakage only gradually. With noise twice the per-dimension standard deviation ($\sigma = 2$), the AUC is still 0.72 and 0.69, while GraphSAGE accuracy drops by 1.4 and 0.2 points. Simple perturbation is therefore not a sufficient defense. Saved embeddings should not be released and should be deleted once the GNNs are trained. Formal protection requires differentially private training (**?**), which we leave to future work.

8. Discussion

This section interprets the experimental evidence, states the theoretical and practical implications of DIVERGE, and clarifies how the proposed framework differs from representative prior TAG methods summarized in Table 1.

8.1. Interpretation of main findings

The results in Tables 4 and 5 support a single overarching claim: *representation diversity induced by LoRA initialization, rather than textual augmentation, is sufficient to outperform augmentation-heavy LLM–GNN pipelines when node texts are already informative*. On all five benchmarks, DIVERGE achieves the best reported accuracy and Macro-F1 among compared methods, including classical GNNs, frozen- and fine-tuned-language-model encoders, and hybrid approaches such as TAPE, ENGINE, and LLaGA. Relative to the strongest augmentation-based baseline (TAPE), DIVERGE improves accuracy by 1.31–5.68 percentage points across datasets, with the largest gains on Cora and CiteSeer, where topic boundaries are fine-grained and class imbalance is more pronounced.

Three mechanisms explain these improvements. First, different LoRA initialization strategies produce semantically distinct node embeddings despite identical training texts and labels, as evidenced by the t-SNE visualizations in Figure 2 and the heterogeneous per-class error profiles in Table 16. Each initialization captures a complementary semantic view of the same nodes, so the resulting GNN pool is genuinely diverse rather than redundant. Second, trainable ensemble learning exploits this diversity more effectively than uniform averaging: all three proposed aggregation schemes outperform the Uniform baseline (Table 13), and class-level weighting reduces errors on minority classes that individual GNNs misclassify. Third, LLM fine-tuning itself yields statistically significant downstream gains, as confirmed by McNemar's test on four datasets (Table 17), indicating that the encoder stage materially improves GNN inputs rather than merely recombining fixed representations.

Together, these findings answer the motivating question posed in Section 4: competitive performance *can* be achieved without textual augmentation when diversity is induced directly from native node texts through multiple parameter-efficient fine-tuning trajectories and adaptively ensembled GNN predictions.

8.2. Theoretical implications

From a theoretical standpoint, DIVERGE advances TAG research in three ways.

Diversity from optimization trajectories. Most LLM–GNN pipelines derive diversity from different *modalities* or pipeline stages—for example, augmented versus native text (TAPE), retrieved versus local context (SKETCH), or task-adaptive prompting (LLM-BP). DIVERGE instead treats **LoRA initialization diversity** as a controllable source of complementary semantic subspaces for the same backbone encoder. This extends parameter-efficient fine-tuning theory (Hu et al., 2021; Meng et al., 2024; Paischer et al., 2025) to graph learning and shows that multiple low-rank adaptation trajectories can explore distinct local minima even under identical node texts and labels.

Limits of textual augmentation. The preliminaries in Section 4 and the Cora feature comparison in Table 2 demonstrate that LLM-generated augmentations can *degrade* performance when native texts are already rich. DIVERGE provides empirical confirmation at scale: it surpasses TAPE on all five benchmarks without generating explanations, summaries, or pseudo-labels. Theoretically, this suggests that augmentation should be treated as a conditional strategy—beneficial when texts are sparse or noisy, but potentially harmful when native semantics are sufficient—rather than as a default prerequisite for LLM–GNN integration.

Adaptive aggregation over heterogeneous learners. Uniform logit averaging assumes equal utility across models and classes, an assumption violated when different initializations specialize on different label groups (Table 16). DIVERGE's learned model- and class-level weights formalize a more flexible fusion principle: ensemble coefficients should be data-dependent and class-aware. This aligns with broader ensemble-learning theory while offering a concrete, trainable alternative to fixed averaging in TAG pipelines.

Finally, the results corroborate the LLMNodeBed finding that strong *local* encoders are often more reliable than costly enhancers (Wu et al., 2025). DIVERGE operationalizes this insight by investing compute in diverse encoder trajectories rather than in external API-based text generation.

8.3. Positioning relative to prior work

Table 1 summarizes the design differences; here we connect those differences to the empirical outcomes reported in Section 7.

Versus TAPE (He et al., 2024): TAPE enriches node text via LLM explanations and zero-shot predictions, then uniformly averages multiple GNNs. DIVERGE uses native text only, induces diversity through multi-initialization

LoRA, and learns ensemble weights. DIVERGE outperforms TAPE on every benchmark despite omitting augmentation and API calls, supporting the claim that initialization-driven diversity can substitute for generated text when node descriptions are informative.

Versus UltraTAG (Zhang et al., 2026): UltraTAG modularizes augmentation, encoding, and training into a configurable pipeline that may still rely on optional external services. DIVERGE offers a narrower but more deployable encoder–ensemble design with a single diversity mechanism (LoRA initialization) and no augmentation module, achieving stronger results on the LLMNodeBed benchmarks used here.

Versus LLM-BP (Wang et al., 2025a): LLM-BP introduces diversity through task-adaptive prompting and fuses information via belief propagation on a single embedding space. DIVERGE derives diversity from multiple fine-tuning trajectories of one backbone and fuses GNN logits through learned ensemble weights. The two approaches are complementary in philosophy—graph-adaptive propagation versus heterogeneous-model ensembling—and DIVERGE's gains on the shared benchmarks suggest that multi-trajectory encoding merits equal consideration alongside task-adaptive single encoders.

Versus SKETCH (Zhou et al., 2025): SKETCH retrieves related texts and distant structural contexts to enrich LM inputs and reduces reliance on conventional GNN propagation. DIVERGE stays GNN-centric and uses only each node's native text, offering a complementary design point: diversity from encoder initialization rather than from retrieved cross-node context. Both methods avoid paid API augmentation, but DIVERGE targets settings where local node text is informative and graph propagation through GNNs remains desirable.

Versus STAGE (Zolnai-Lucas et al., 2024): STAGE ensembles GNNs trained on embeddings from a *single* frozen LLM. DIVERGE goes further by generating multiple adapted encoders through LoRA and learning non-uniform combination weights, explaining its consistent accuracy and Macro-F1 advantages.

In summary, no prior TAG method simultaneously (i) induces representation diversity through multi-initialization LoRA on native node text, (ii) trains a pool of GNNs on complementary embeddings, and (iii) fuses predictions with learned model- and class-level weights—all without textual augmentation or external APIs. DIVERGE fills this gap empirically on five standard benchmarks and conceptually by reframing diversity as an encoder-side, initialization-driven property rather than an augmentation-side artifact.

9. Limitations

Despite the strong performance of the proposed DIVERGE framework, several limitations remain.

First, the proposed method demonstrates stronger performance in fully supervised settings compared to semi-supervised scenarios. The primary reason is that the adaptive ensemble learning framework relies on sufficient labeled data in order to effectively learn the contribution and weighting of different models. In semi-supervised settings, where only a limited number of labeled nodes are available, the ensemble model may not accurately estimate the optimal combination coefficients, which can reduce the overall effectiveness of the framework.

Second, although the proposed approach eliminates the need for costly textual augmentation and external LLM API calls, it does not fully exploit all capabilities of LLMs. In this work, LLMs are mainly utilized as semantic encoders for extracting node-level textual representations. However, recent studies have shown that LLMs can also play important roles as graph enhancers, for example through structural refinement, semantic edge generation, textual augmentation, or label enhancement. These capabilities were not extensively explored in the current framework.

Another limitation is the computational overhead associated with training multiple LLMs and GNNs using different initialization strategies. Although parameter-efficient fine-tuning methods such as LoRA significantly reduce the number of trainable parameters, the overall training process still requires multiple independent optimization stages and additional memory resources for storing several models simultaneously.

Finally, the proposed framework was mainly evaluated on citation-based text-attributed graphs containing relatively rich textual information. Therefore, the generalization ability of the method to graphs with short, noisy, or low-quality texts (such as social media graphs or recommendation systems) remains an open research question.

Future work can address these limitations by designing more robust ensemble mechanisms for low-label regimes, integrating structural and textual enhancement capabilities of LLMs, reducing the computational complexity of multi-model training, and extending the framework to broader categories of text-attributed graphs.

10. Conclusion

In this paper, we proposed **DIVERGE**, a novel framework for learning on text-attributed graphs that combines LLMs and GNNs through diverse representation learning and adaptive ensemble mechanisms. Unlike previous approaches that rely heavily on expensive textual augmentation or external LLM APIs, the proposed method demonstrates that diverse and complementary node representations can be effectively generated solely from the original graph texts by utilizing multiple LoRA fine-tuning strategies with different parameter initializations.

To achieve this objective, multiple parameter-efficient LLMs were independently fine-tuned using different initialization methods, and the extracted textual representations were used to train multiple GNNs. Subsequently, an adaptive ensemble learning framework was introduced to combine the outputs of these graph models through trainable weighting mechanisms at both the model and class levels. Experimental results on several benchmark text-attributed graph datasets, including Cora, CiteSeer, PubMed, WikiCS, and OGBN-ArXiv, demonstrated that the proposed framework consistently outperforms existing GNN-based and hybrid LLM-GNN baselines in terms of Accuracy and Macro-F1 metrics.

Furthermore, the experimental analysis showed that different LoRA initialization strategies produce semantically distinct and complementary embeddings. The obtained results also confirmed that trainable ensemble learning methods provide superior performance compared to simple averaging strategies by more effectively exploiting the diversity among representations. In addition, statistical analysis using McNemar's test demonstrated that fine-tuning the LLM leads to statistically significant improvements in downstream graph learning performance.

Overall, the findings of this research suggest that representation diversity itself can serve as an effective alternative to costly textual augmentation for improving learning on text-attributed graphs. By relying on locally fine-tuned encoders rather than external LLM APIs, DIVERGE also offers a deployment path for graphs whose node text cannot leave organizational or regulatory boundaries; because the saved embeddings and adapters can still reveal the text, they must be protected like the raw data (Section 7.8.6). The proposed framework opens new directions for integrating parameter-efficient LLM fine-tuning, representation diversity, and adaptive ensemble learning in graph-based machine learning systems.

Data availability

Benchmark datasets are available via ([dataset] Wu et al., 2025) (accessed 6 June 2026). Source code for DIVERGE is publicly available at `https://github.com/nimaiparifard/DIVERGE`.

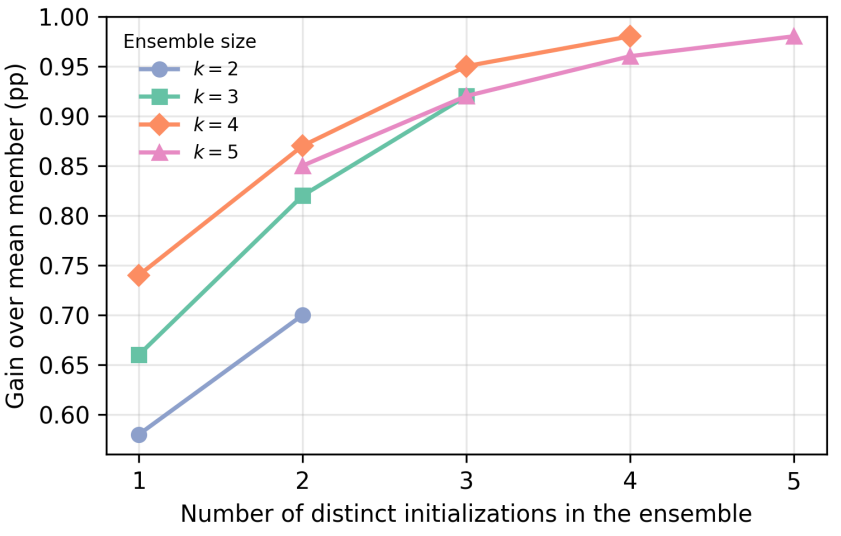


Figure 3: Ensemble gain over the mean member as a function of the number of distinct initializations among the k members, averaged over all k-member subsets of the CiteSeer pool (Gemma-2-2B, uniform averaging).

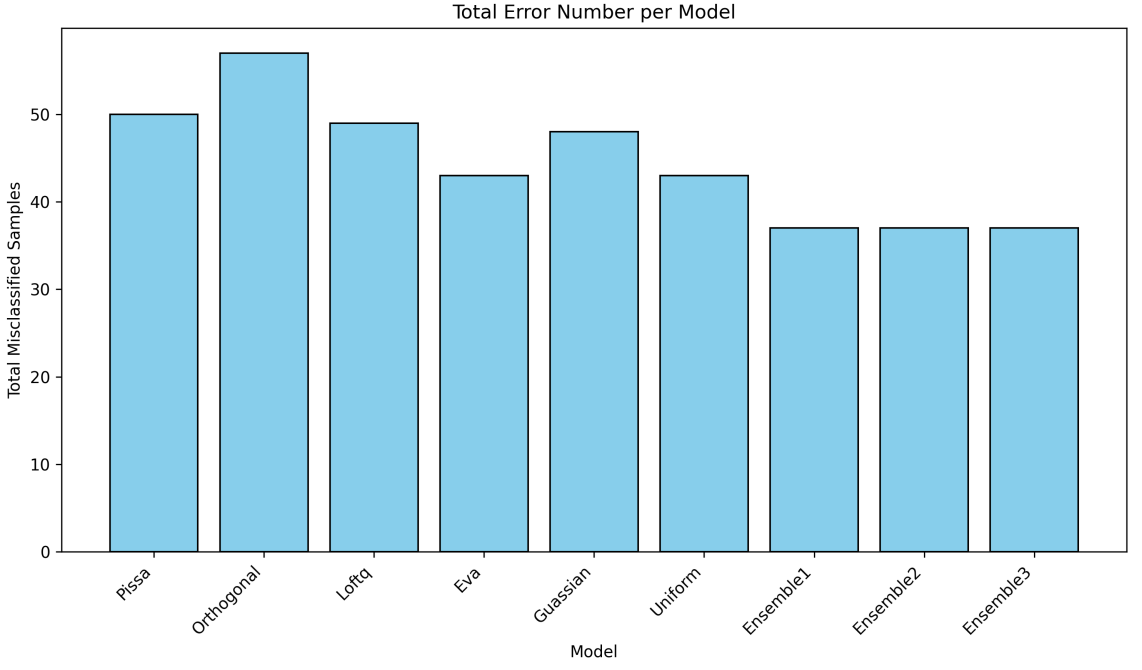


Figure 4: Comparison of the total number of classification errors between individual models and ensemble learning methods.

The reduction in the number of errors in ensemble methods demonstrates the effective utilization of the diversity in textual representations derived from different initializations.

References

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020. Language models are few-shot learners, in: Proceedings of the 34th International Conference on Neural Information Processing Systems, p. 1877–1901. doi:doi.org/10.5555/3495724.3495883.

Cai, T.T., Ma, R., 2022. Theoretical foundations of t-sne for visualizing high-dimensional clustered data. Journal of Machine Learning Research 23, 1–54. doi:doi.org/10.5555/3586589.3586890.

Chen, R., Zhao, T., Jaiswal, A., Shah, N., Wang, Z., 2024a. LLaGA: Large language and graph assistant, in: Proceedings of the 41st International Conference on Machine Learning (ICML), pp. 7809–7823. doi:doi.org/10.5555/3692070.3692376.

Chen, Z., Mao, H., Li, H., Jin, W., Wen, H., Wei, X., Wang, S., Yin, D., Fan, W., Liu, H., Tang, J., 2024b. Exploring the potential of large language models (llms) in learning on graphs. SIGKDD Explorations Newsletter 25, 42–61. doi:doi.org/10.1145/3655103.3655110.

Devlin, J., Chang, M.W., Lee, K., Toutanova, Kristina Burstein, J., Doran, C., Solorio, T., . BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. doi:doi.org/10.18653/v1/N19-1423.

Giles, C.L., Bollacker, K.D., Lawrence, S., 1998. Citeseer: An automatic citation indexing system, in: Proceedings of the Third ACM Conference on Digital Libraries, pp. 89–98. doi:doi.org/10.1145/276675.276685.

Guo, Z., Xia, L., Yu, Y., Wang, Y., Lu, K., Huang, Z., Huang, C., 2025. Graphedit: Large language models for graph structure learning URL: https://arxiv.org/abs/2402.15183, arXiv:2402.15183.

Hamilton, W.L., Ying, R., Leskovec, J., 2017. Inductive representation learning on large graphs, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, pp. 1025–1035. doi:doi.org/10.5555/3294771.3294869.

He, X., Bresson, X., Laurent, T., Perold, A., LeCun, Y., Hooi, B., 2024. Harnessing explanations: Llm-to-lm interpreter for enhanced text-attributed graph representation learning. URL: https://arxiv.org/abs/2305.19523, arXiv:2305.19523.

Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., Gelly, S., 2019. Parameter-efficient transfer learning for nlp. URL: https://arxiv.org/abs/1902.00751, arXiv:1902.00751.

Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., 2021. Lora: Low-rank adaptation of large language models URL: https://arxiv.org/abs/2106.09685, arXiv:2106.09685.

Hu, L., Xie, H., Yu, L., Huang, T., Li, L., Li, M., ZHOU, J., Wang, D., 2024. Low-cost enhancer for text attributed graph learning via graph alignment. URL: https://openreview.net/forum?id=yrnrvfXFaV.

Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., Leskovec, J., 2020. Open graph benchmark: Datasets for machine learning on graphs, in: Proceedings of the 34th International Conference on Neural Information Processing Systems, pp. 22118–22133. doi:doi.org/10.5555/3495724.3497579.

Huang, X., Han, K., Yang, Y., Bao, D., Tao, Q., Chai, Z., Zhu, Q., 2024. GNNs as adapters for LLMs on text-attributed graphs, in: Proceedings of The Web Conference (WWW), pp. 893–904. doi:doi.org/10.1145/3589334.3645627.

Jaiswal, A., Choudhary, N., Adkathimar, R., Alagappan, M.P., Hiranandani, G., Ding, Y., Wang, Z., Huang, E.W., Subbian, K., 2024. All against some: Efficient integration of large language models for message passing in graph neural networks. URL: https://arxiv.org/abs/2407.14996, arXiv:2407.14996.

Jin, B., Liu, G., Han, C., Jiang, M., Ji, H., Han, J., 2024. Large language models on graphs: A comprehensive survey. IEEE Transactions on Knowledge and Data Engineering 36, 8622–8642. doi:doi.org/10.1109/tkde.2024.3469578/mm1.

Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks, in: Proceedings of the 5th International Conference on Learning Representations. URL: https://openreview.net/forum?id=SJU4ayYgl.

Li, Y., Li, Z., Wang, P., Li, J., Sun, X., Cheng, H., Yu, J.X., 2024a. A survey of graph meets large language model: Progress and future directions, in: Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI), pp. 8123 – 8131. doi:doi.org/10.24963/ijcai.2024/898.

Li, Y., Wang, P., Li, Z., Yu, J.X., Li, J., 2024b. ZeroG: Investigating cross-dataset zero-shot transferability in graphs, in: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 1725–1735. doi:doi.org/10.1145/3637528.3671982.

Li, Y., Yu, Y., Liang, C., He, P., Karampatziakis, N., Chen, W., Zhao, T., 2023. Loftq: Lora-fine-tuning-aware quantization for large language models URL: https://arxiv.org/abs/2310.08659, arXiv:2310.08659.

Liu, H., Feng, J., Kong, L., Liang, N., Tao, D., Chen, Y., Zhang, M., 2024. One for all: Towards training one graph model for all classification tasks. arXiv preprint arXiv:2310.00149.

McCallum, A.K., Nigam, K., Rennie, J., Seymore, K., 2000. Automating the construction of internet portals with machine learning. Information Retrieval 3, 127–163. doi:doi.org/10.1023/a:1009953814988.

Meng, F., Wang, Z., Zhang, M., 2024. Pissa: Principal singular values and singular vectors adaptation of large language models, in: Proceedings of the 38th International Conference on Neural Information Processing Systems, pp. 121038 – 121072. doi:doi.org/10.52202/079017-3846.

Mernyei, P., Cangea, C., 2022. Wiki-cs: A wikipedia-based benchmark for graph neural networks URL: https://arxiv.org/abs/2007.02901, arXiv:2007.02901.

Paischer, F., Hauzenberger, L., Schmied, T., Alkin, B., Deisenroth, M.P., Hochreiter, S., 2025. Parameter efficient fine-tuning via explained variance adaptation URL: https://arxiv.org/abs/2410.07170, arXiv:2410.07170.

Qin, Y., Wang, X., Zhang, Z., Zhu, W., 2024. Disentangled representation learning with large language models for text-attributed graphs URL: https://arxiv.org/abs/2310.18152, arXiv:2310.18152.

Ramírez Molina, A.A., Li, S., Gong, J., 2026. Mapping scholarly knowledge: A systematic review of knowledge graphs for academic papers. Information Processing Management 63, 104842. doi:doi.org/10.1016/j.ipm.2026.104842.

Reimers, N., Gurevych, I., 2019. Sentence-bert: Sentence embeddings using siamese bert-networks, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992. doi:doi.org/10.18653/v1/d19-1410.

Sen, P., Namata, G., Bilgic, M., Getoor, L., Gallagher, B., Eliassi-Rad, T., 2008. Collective classification in network data. AI Magazine 29, 93–106. doi:doi.org/10.1609/aimag.v29i3.2157.

Sun, S., Ren, Y., Chen, J., Ma, C., 2026. Large language models as topological structure enhancers for text-attributed graphs, in: Proceedings of the 30th International Conference on Database Systems for Advanced Applications (DASFAA), pp. 106–122. doi:doi.org/10.1007/978-981-95-4158-4_7.

Tan, Y., Zhou, Z., Lv, H., Liu, W., Yang, C., 2023. WalkLM: A uniform language model fine-tuning framework for attributed graph embedding, in: Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS), pp. 13308 – 13325. doi:doi.org/10.52202/075280-0584.

Tang, J., Yang, Y., Wei, W., Shi, L., Su, L., Cheng, S., Yin, D., Huang, C., 2024. Graphgpt: Graph instruction tuning for large language models, in: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 491–500. doi:doi.org/10.1145/3626772.3657775.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., Lample, G., 2023. Llama: Open and efficient foundation language models URL: https://arxiv.org/abs/2302.13971, arXiv:2302.13971.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph attention networks. URL: https://arxiv.org/abs/1710.10903, arXiv:1710.10903.

Wang, H., Liu, S., Wei, R., Li, P., 2025a. Generalization principles for inference over text-attributed graphs with large language models, in: Proceedings of the 42nd International Conference on Machine Learning, pp. 63755 – 63776. doi:doi.org/10.5555/3780338.3782868.

Wang, J., Wang, J., Jin, D., 2026. Disentangled graph recommendation via dynamic long and short-term intent modeling. Information Processing Management 63, 104786. doi:doi.org/10.1016/j.ipm.2026.104786.

Wang, L., Chen, S., Jiang, L., Pan, S., Cai, R., Yang, S., Yang, F., 2025b. Parameter-efficient fine-tuning in large language models: A survey of methodologies. Artificial Intelligence Review 58, 227. doi:doi.org/10.1007/s10462-025-11236-4.

Wu, X., Shen, Y., Ge, F., Shan, C., Jiao, Y., Sun, X., Cheng, H., 2025. When do llms help with node classification? a comprehensive analysis, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), pp. 28748–28760. doi:doi.org/10.52202/075280-1251.

[dataset] Wu, X., Shen, Y., Ge, F., Shan, C., Jiao, Y., Sun, X., Cheng, H., 2025. When do llms help with node classification? a comprehensive analysis. https://huggingface.co/datasets/xxwu/LLMNodeBed. Data repository: Hugging Face Datasets. License: MIT. Accessed 6 June 2026.

Xie, H., Hu, L., Yu, L., Huang, T., Li, L., Li, M., Zhou, J., Wang, H., Wang, D., 2025. Efficient text-attributed graph learning through selective annotation and graph alignment URL: https://arxiv.org/abs/2506.07168, arXiv:2506.07168.

Xue, R., Shen, X., Yu, R., Liu, X., 2025. Efficient end-to-end language model fine-tuning on graphs, in: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), pp. 3449–3460. doi:doi.org/10.1145/3711896.3736922.

Yu, J., Ren, Y., Gong, C., Tan, J., Li, X., Zhang, X., 2025. Leveraging large language models for node generation in few-shot learning on text-attributed graphs, in: Proceedings of the 39th AAAI Conference on Artificial Intelligence (AAAI), pp. 13087 – 13095. doi:doi.org/10.1609/aaai.v39i12.33428.

Zhang, D.C., Yang, M., Ying, R., Lauw, H.W., 2024. Text-attributed graph representation learning: Methods, applications, and challenges, in: Companion Proceedings of the ACM Web Conference (WWW), Singapore, Singapore. pp. 1298–1301. doi:doi.org/10.1145/3589335.3641255.

Zhang, T., Yang, R., Lai, Y., Yan, M., Ye, X., Fan, D., 2025a. Leveraging large language models for effective label-free node classification in text-attributed graphs, in: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 698 – 708. doi:doi.org/10.1145/3726302.3730021.

Zhang, X., Chen, G.Z., Li, S., Liu, Z., Chen, C.L.P., Zhang, T., 2025b. An orthogonal high-rank adaptation for large language models, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 18815–18833. doi:doi.org/10.18653/v1/2025.emnlp-main.951.

Zhang, Z., Li, X., Li, R.H., Li, Z., Zhou, B., Wang, G., 2026. Toward general and robust llm-enhanced text-attributed graph learning, in: Proceedings of the 2026 International Conference on Multimedia Retrieval, p. 1477–1485. doi:10.1145/3805622.3810684.

Zhao, J., Zhuo, L., Shen, Y., Qu, M., Liu, K., Bronstein, M., Zhu, Z., Tang, J., 2023. Graphtext: Graph reasoning in text space. URL: https://arxiv.org/abs/2310.01089, arXiv:2310.01089.

Zhou, C., Wang, Z., Chen, S., Du, J., Zheng, Q., Xu, Z., Huang, X., 2025. Taming language models for text-attributed graph learning with decoupled aggregation, in: Che, W., Nabende, J., Shutova, E., Pilehvar, M.T. (Eds.), Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics. pp. 3463–3474. doi:doi.org/10.18653/v1/2025.acl-long.173.

Zhu, Y., Wang, Y., Shi, H., Tang, S., 2024. Efficient tuning and inference for large language models on textual graphs, in: Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI), pp. 5734 – 5742. doi:doi.org/10.24963/ijcai.2024/634.

Zhuang, L., Wayne, L., Ya, S., Jun, Z., 2021. A robustly optimized bert pre-training approach with post-training, in: Proceedings of the 20th Chinese National Conference on Computational Linguistics, pp. 1218–1227. doi:doi.org/10.1007/978-3-030-84186-7_31.

Zolnai-Lucas, A., Boylan, J., Hokamp, C., Ghaffari, P., 2024. STAGE: Simplified text-attributed graph embeddings using pre-trained LLMs, in: Biswas, R., Kaffee, L.A., Agarwal, O., Minervini, P., Singh, S., de Melo, G. (Eds.), Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models (KaLLM 2024), pp. 92–104. doi:doi.org/10.18653/v1/2024.kallm-1.10.

Table 25
LLM and LoRA hyperparameters of the three LLM encoders (top) and the per-dataset LLM training schedule (bottom).

Setting	LLaMA-3.2-1B	SmolLM2-1.7B	Gemma-2-2B
Parameters / hidden size / layers	1.24B / 2048 / 16	1.71B / 2048 / 24	2.61B / 2304 / 26
Loading precision	bfloat16	4-bit	4-bit
Trainable LoRA parameters	1.72M	3.16M	3.21M
Peak fine-tuning VRAM	3.9 GB	4.8 GB	7.3 GB
Embedding caching batch size	32	8	4
Shared by the three encoders			
LoRA rank (r) / alpha	8 / 16, rank-stabilized scaling (**?**)		
LoRA target modules	Query, key, value and output projections; classification head fully trained		
LoRA dropout	0.05		
LoRA initialization	Gaussian, Orthogonal, PiSSA, EVA, LoFTQ ($M = 5$ runs)		
Learning rate / scheduler	2×10^{-4}, linear decay with 3% warm-up		
Batch size	16 (per-device batch 1, gradient accumulation 16)		
Max sequence length	512 tokens		
Checkpoint selection	Best validation accuracy		
Node embedding	Mean of the last hidden layer over non-padding tokens		

Schedule	Cora	CiteSeer	PubMed	OGBN-ArXiv	WikiCS	Reddit	Computers
LLM epochs	3	3	2	1	2	1	1
Validation every (steps)	50	50	500	10,000	500	10,000	10,000

Table 26
Search spaces of the GNN encoders and of the ensemble heads. Both are searched by exhaustive grid search on validation accuracy, separately for every LLM encoder, GNN encoder and dataset.

Hyperparameter	Values
GNN encoders (GCN, GIN, GraphSAGE): 512 configurations	
Learning rate	$\{10^{-2}, 5 \times 10^{-3}, 10^{-3}, 10^{-4}\}$
Hidden dimension	$\{128, 256, 512, 1024\}$
Layers	$\{2, 3\}$
Dropout	$\{0.0, 0.01, 0.1, 0.5\}$
Weight decay	$\{0.0, 0.01\}$
Batch normalization	{no, yes}
Training	Adam, max. 200 epochs, early stopping (patience 20)
Selection	Top 64 configurations re-run with 5 seeds
Ensemble heads: 36 configurations	
MLP layers	$\{2, 3\}$
Hidden size	$\{64, 128, 256\}$
Dropout	$\{0.3, 0.5\}$
Learning rate	$\{10^{-4}, 10^{-3}, 10^{-2}\}$
Training	Adam, 200 epochs, epoch chosen on validation

A. Hyperparameters

This appendix lists all hyperparameters of DIVERGE. Table 25 gives the LLM and LoRA settings of the three LLM encoders and the per-dataset LLM training schedule, Table 26 the search spaces of the GNN encoders and the ensemble heads, and Tables 27–29 the GNN hyperparameters selected for every LLM encoder, GNN encoder and dataset.

Table 27
Selected GNN hyperparameters with the LLaMA-3.2-1B encoder, per GNN encoder and dataset (grid search on validation accuracy, Table 26).

Hyperparameter	Cora	CiteSeer	PubMed	OGBN-ArXiv	WikiCS	Reddit	Computers
GCN							
Layers	2	3	3	2	2	2	2
Hidden dim	1024	128	256	512	1024	1024	128
Learning rate	1×10^{-4}	1×10^{-3}	5×10^{-3}	1×10^{-4}	0.01	0.01	1×10^{-3}
Dropout	0.0	0.1	0.01	0.5	0.1	0.01	0.0
Weight decay	0.0	0.0	0.01	0.0	0.0	0.0	0.0
Batch norm	yes	no	yes	no	yes	yes	no
GIN							
Layers	2	2	2	3	3	2	3
Hidden dim	128	128	512	256	128	128	128
Learning rate	1×10^{-3}	0.01	1×10^{-4}	5×10^{-3}	1×10^{-4}	0.01	0.01
Dropout	0.0	0.0	0.5	0.01	0.1	0.01	0.0
Weight decay	0.0	0.0	0.01	0.0	0.01	0.01	0.01
Batch norm	yes	yes	no	yes	yes	yes	yes
GraphSAGE							
Layers	3	3	3	2	3	3	3
Hidden dim	256	128	256	128	512	512	1024
Learning rate	5×10^{-3}	1×10^{-3}	0.01	0.01	1×10^{-4}	1×10^{-3}	5×10^{-3}
Dropout	0.5	0.5	0.5	0.1	0.1	0.0	0.5
Weight decay	0.0	0.01	0.01	0.0	0.01	0.01	0.01
Batch norm	yes	yes	no	yes	yes	yes	yes

Table 28
Selected GNN hyperparameters with the SmolLM2-1.7B encoder, per GNN encoder and dataset (grid search on validation accuracy, Table 26).

Hyperparameter	Cora	CiteSeer	PubMed	OGBN-ArXiv	WikiCS	Reddit	Computers
GCN							
Layers	2	3	2	3	2	2	3
Hidden dim	512	128	256	1024	256	1024	512
Learning rate	1×10^{-3}	1×10^{-3}	0.01	5×10^{-3}	1×10^{-3}	0.01	1×10^{-3}
Dropout	0.1	0.1	0.0	0.5	0.1	0.1	0.1
Weight decay	0.01	0.0	0.0	0.01	0.01	0.0	0.01
Batch norm	no	no	yes	no	yes	yes	no
GIN							
Layers	2	2	3	2	3	2	2
Hidden dim	128	128	256	1024	128	128	128
Learning rate	0.01	0.01	5×10^{-3}	1×10^{-4}	1×10^{-3}	5×10^{-3}	1×10^{-3}
Dropout	0.01	0.1	0.0	0.5	0.01	0.0	0.01
Weight decay	0.0	0.0	0.01	0.0	0.0	0.0	0.0
Batch norm	yes	yes	no	yes	yes	yes	yes
GraphSAGE							
Layers	3	3	2	3	2	3	3
Hidden dim	128	512	256	128	512	512	512
Learning rate	0.01	1×10^{-4}	1×10^{-3}	0.01	1×10^{-4}	1×10^{-4}	1×10^{-4}
Dropout	0.0	0.5	0.1	0.01	0.0	0.1	0.0
Weight decay	0.01	0.01	0.0	0.01	0.01	0.01	0.01
Batch norm	no	no	yes	no	yes	yes	no

Table 29
Selected GNN hyperparameters with the Gemma-2-2B encoder, per GNN encoder and dataset (grid search on validation accuracy, Table 26).

Hyperparameter	Cora	CiteSeer	PubMed	OGBN-ArXiv	WikiCS	Reddit	Computers
GCN							
Layers	2	2	3	2	2	3	2
Hidden dim	128	128	512	256	512	128	1024
Learning rate	1×10^{-3}	5×10^{-3}	1×10^{-4}	0.01	5×10^{-3}	5×10^{-3}	1×10^{-3}
Dropout	0.5	0.0	0.1	0.01	0.5	0.0	0.5
Weight decay	0.01	0.0	0.01	0.0	0.0	0.01	0.0
Batch norm	yes	yes	no	yes	yes	no	yes
GIN							
Layers	2	2	2	3	2	2	3
Hidden dim	512	512	128	512	256	256	1024
Learning rate	1×10^{-3}	5×10^{-3}	0.01	1×10^{-3}	1×10^{-4}	1×10^{-4}	5×10^{-3}
Dropout	0.0	0.01	0.5	0.0	0.0	0.01	0.1
Weight decay	0.0	0.0	0.0	0.01	0.0	0.0	0.01
Batch norm	yes	yes	no	yes	yes	no	yes
GraphSAGE							
Layers	3	2	3	2	3	3	2
Hidden dim	128	128	512	256	256	128	1024
Learning rate	1×10^{-3}	1×10^{-4}	5×10^{-3}	1×10^{-3}	1×10^{-3}	0.01	1×10^{-4}
Dropout	0.5	0.5	0.01	0.0	0.5	0.1	0.5
Weight decay	0.0	0.01	0.0	0.01	0.0	0.0	0.01
Batch norm	yes	no	yes	no	yes	yes	no